



Facultade de Administración e Dirección de Empresas

**Traballo de
Fin de Grao**

Análise experimental da
optimización de
sistemas RAG mediante
técnicas de recuperación
e procesamento de
contexto

Roi Caride Borrajo

Xuño 2026

Resumo

Os sistemas de *Retrieval-Augmented Generation* (RAG) emerxeron como un dos paradigmas máis prometedores para dotar os modelos de linguaxe de gran escala de coñecemento factual actualizado e verificable. Ao integrar un módulo de recuperación de información sobre fontes externas, os sistemas RAG mitigan dúas das limitacións máis críticas dos LLM: a tendencia a xerar contido incorrecto (alucinación) e a obsolescencia do coñecemento derivada do corte temporal do adestramento.

Non obstante, o rendemento dun sistema RAG non depende unicamente da capacidade xerativa do modelo de linguaxe, senón en igual ou maior medida das decisións de deseño en cada etapa da pipeline: que modelo de embedding se usa para a indexación, como se segmentan os documentos, cales son os mecanismos de recuperación e como se filtra o contexto antes de ser entregado ao xerador.

Este traballo presenta unha análise experimental sistemática mediante un deseño de ablación secuencial en cinco fases: (0) establecemento dunha liña base, (1) comparativa de modelos de embedding, (2) comparativa de estratexias de segmentación, (3) comparativa de técnicas de retrieval incluíndo recuperación híbrida, (4) avaliación de técnicas de expansión de consultas (HyDE, reescrita, multi-query, RAG-Fusion, entre outras) e (5) avaliación de reranking pos-recuperación. O corpus está composto por documentos xurídicos reais procedentes de EUR-Lex, e o rendemento mídese con seis métricas do framework RAGAS: context recall, context precision, context entity recall, noise sensitivity, faithfulness e response relevancy. O traballo achega dous resultados complementarios: a configuración empírica óptima para o dominio xurídico e, sobre todo, un **marco experimental reproducible** que permite determinar a configuración axeitada de calquera sistema RAG no seu propio dominio, un activo metodolóxico de aplicabilidade directa en contornos profesionais.

Palabras clave: RAG, modelos de linguaxe de gran escala, recuperación de información, embeddings, chunking, retrieval híbrido, reranking, HyDE, EUR-Lex, RAGAS.

Número de palabras: 10.630

Abstract

Retrieval-Augmented Generation (RAG) systems have emerged as one of the most promising paradigms for equipping large language models with up-to-date and verifiable factual knowledge. By integrating an information retrieval module over external sources, RAG systems mitigate two critical limitations of LLMs: the tendency to generate incorrect content (hallucination) and knowledge obsolescence derived from the training data cut-off.

However, the performance of a RAG system depends not only on the generative capability of the language model but equally on design decisions at each stage of the pipeline: which embedding model is used for indexing, how documents are segmented, what retrieval mechanisms are employed, and how context is filtered before being delivered to the generator.

This work presents a systematic experimental analysis through a sequential ablation design in five optimization phases over an honest baseline: (1) a comparison of embedding models, (2) a comparison of chunking strategies, (3) a comparison of retrieval techniques including hybrid retrieval, (4) an evaluation of query expansion techniques (HyDE, rewriting, multi-query, RAG-Fusion, among others), and (5) an evaluation of post-retrieval reranking; the resulting pipeline is finally validated against a state-of-the-art stack from the literature. The corpus consists of real legal documents from EUR-Lex, and performance is measured with six RAGAS metrics: context recall, context precision, context entity recall, noise sensitivity, faithfulness, and response relevancy. This work delivers two complementary contributions: the empirically optimal configuration for the legal domain and, above all, a **reproducible experimental framework** that enables determining the right configuration of any RAG system in its own domain, a methodological asset of direct applicability in professional settings.

Keywords: RAG, large language models, information retrieval, embeddings, chunking, hybrid retrieval, reranking, HyDE, EUR-Lex, RAGAS.

Índice xeral

Resumo	i
Abstract	ii
Índice de abreviaturas	v
Introdución	1
Planificación	3
Desenvolvemento do traballo	4
1. Estado da arte e marco teórico	4
1.1. Modelos de linguaxe de gran escala	4
1.2. Retrieval-Augmented Generation: fundamentos	6
1.3. Compoñentes dunha pipeline RAG	7
1.4. Técnicas avanzadas de optimización	9
1.5. Frameworks de avaliación para RAG	11
1.6. Framework de pipelines: Kedro	14
1.7. Revisión de traballos relacionados	14
2. Corpus documental	15
2.1. EUR-Lex como fonte de datos	15
2.2. Criterios de selección e características do corpus	15
2.3. Preprocesamento e limpeza	17
2.4. Selección do corpus de traballo	18
2.5. Construción do golden dataset de avaliación	19
3. Metodoloxía experimental	20
3.1. Preguntas de investigación	20
3.2. Deseño experimental	20
3.3. Arquitectura do sistema experimental	21
3.4. Técnicas avaliadas	23
3.5. Métricas de avaliación	26
4. Resultados e discusión	27
4.1. Experimento 1: modelos de embedding	27
4.2. Experimento 2: estratexias de chunking	28
4.3. Experimento 3: técnicas de retrieval	29
4.4. Experimento 4: expansión e transformación de consultas	30

4.5.	Experimento 5: reranking pos-recuperación	31
4.6.	Progresión global e pipeline final	32
4.7.	Validación contra un <i>stack</i> estado da arte	34
Conclusións e ampliación		35
Bibliografía		39

Índice de abreviaturas

RAG	Retrieval-Augmented Generation
LLM	Large Language Model
NLP	Natural Language Processing
IR	Information Retrieval
QA	Question Answering
API	Application Programming Interface
EU	European Union
MRR	Mean Reciprocal Rank
NDCG	Normalized Discounted Cumulative Gain
HyDE	Hypothetical Document Embeddings
MMR	Maximal Marginal Relevance
DPR	Dense Passage Retrieval
BERT	Bidirectional Encoder Representations from Transformers
GPT	Generative Pre-trained Transformer
BM25	Best Match 25
ANN	Approximate Nearest Neighbour
PDF	Portable Document Format
BGE	BAAI General Embedding
RRF	Reciprocal Rank Fusion
CRAG	Corrective Retrieval-Augmented Generation
TF-IDF	Term Frequency–Inverse Document Frequency

Índice de cadros

1.	Planificación temporal do traballo	3
2.	Composición do corpus EUR-Lex por familia de documentos	16
3.	Percentís de lonxitude por familia de documentos (palabras)	17
4.	Composición do corpus de traballo (subconxunto de copyright) por familia	19
5.	Pipelines Kedro: responsabilidade e artefactos de entrada e saída	22
6.	Métricas de decisión (P) e de observación (S) por experimento	27
7.	Resultados do Experimento 1 (modelos de embedding)	27
8.	Resultados do Experimento 2 (estratexias de chunking)	28
9.	Resultados do Experimento 3 (técnicas de retrieval)	29
10.	Resultados do Experimento 4 (expansión de consultas)	30
11.	Resultados do Experimento 5 (reranking)	31
12.	Progresión acumulada da ablación secuencial (métrica P+R)	32
13.	Baseline fronte á pipeline final óptima	33
14.	Pipeline empírica fronte ao <i>stack</i> SOTA da literatura	34

Índice de figuras

1.	Distribución do corpus EUR-Lex por familia de documentos	16
2.	Distribución da lonxitude dos documentos por familia (escala logarítmica)	17
3.	Arquitectura do sistema experimental: pipelines Kedro e fluxo de datos	22
4.	Comparativa de modelos de embedding (Experimento 1)	28
5.	Comparativa de estratexias de chunking (Experimento 2)	29
6.	Comparativa de técnicas de retrieval (Experimento 3)	30
7.	Comparativa de técnicas de expansión de consultas (Experimento 4)	31
8.	Comparativa de técnicas de reranking (Experimento 5)	32
9.	Progresión acumulada do P+R ao longo da ablación secuencial	33
10.	Evolución das seis métricas RAGAS ao longo da ablación	33
11.	Comparación da pipeline empírica co <i>stack</i> SOTA e o baseline	34

Introdución

O campo do procesamento de linguaxe natural (Natural Language Processing (NLP)) experimentou nos últimos anos unha transformación sen precedentes impulsada polo desenvolvemento dos modelos de linguaxe de gran escala (Large Language Model (LLM)). Modelos como GPT-3 (Brown et al., 2020), baseados na arquitectura Transformer (Vaswani et al., 2017), demostraron unha capacidade extraordinaria para xerar texto coherente, responder preguntas e razoar sobre tarefas diversas, todo iso a partir dun coñecemento implícito codificado nos seus parámetros durante o proceso de adestramento.

Con todo, estes modelos presentan limitacións inherentes que restrinxen a súa aplicabilidade en escenarios do mundo real. En primeiro lugar, o seu coñecemento está limitado á data de corte do adestramento: un modelo adestrado en datos até 2023 non pode responder con fiabilidade sobre eventos acontecidos posteriormente. En segundo lugar, e máis criticamente, os LLM son propensos ao fenómeno das *alucinacións*: a xeración de afirmacións plausibles pero factualmente incorrectas (Ji et al., 2023). En terceiro lugar, carecer de mecanismos de acceso a fontes verificables dificulta o seu uso en dominios de alta responsabilidade como o xurídico, o médico ou o financeiro.

Os sistemas de *Retrieval-Augmented Generation* (Retrieval-Augmented Generation (RAG)), propostos por Lewis et al. (2020) en 2020, emerxeron como resposta directa a estas limitacións. A idea fundamental é engadir un módulo de recuperación de información que, ante cada consulta dun usuario, busca os fragmentos de texto máis relevantes dentro dun corpus de documentos externo e os proporciona como contexto adicional ao modelo xerador.

Non obstante, a arquitectura RAG introduce o seu propio conxunto de decisións de deseño que inflúen de forma determinante no rendemento final do sistema. Como se dividen os documentos para a indexación? Cal é o modelo de *embedding* máis adecuado para o dominio? Cantos fragmentos recuperar? Deberían reordenarse os resultados antes de pasalos ao modelo? Estas preguntas carecen aínda dunha resposta definitiva, e a literatura recente mostra que as respostas óptimas dependen do dominio, da natureza dos documentos e do tipo de tarefa (Y. Gao et al., 2023).

Obxectivo principal. Cuantificar o impacto das principais técnicas de optimización sobre o rendemento dun sistema RAG nun corpus documental xurídico especializado, identificando as configuracións que producen mellores resultados en cada dimensión de calidade avaliada. Como obxectivo complementario, e dada a natureza do Grao en Empresa e Tecnoloxía, o traballo persegue tamén construír un marco experimental reproducible e transferible que poida reutilizarse para optimizar sistemas RAG noutros dominios, dotando os achados dun valor de aplicación profesional máis alá do caso estudado.

Obxectivos específicos:

OE1. Revisar e sistematizar o estado da arte en arquitecturas RAG, modelos de embedding e técnicas de optimización existentes.

- OE2.** Construír e validar un corpus de avaliación baseado en documentos de EUR-Lex, incluíndo un conxunto de pares pregunta-resposta de referencia con respostas ideais (golden dataset).
- OE3.** Identificar o modelo de embedding e a estratexia de chunking que mellor se adaptan á estrutura e terminoloxía dos documentos xurídicos do corpus.
- OE4.** Avaliar técnicas de retrieval de crecente complexidade: busca vectorial densa, BM25, recuperación híbrida e técnicas avanzadas de reformulación de consultas (HyDE, reescrita, multi-query, self-query).
- OE5.** Avaliar o impacto do reranking pos-recuperación (cross-encoder e LLM listwise) sobre a calidade do contexto e da resposta.
- OE6.** Comparar cuantitativamente todas as configuracións empregando seis métricas do framework RAGAS, e validar a pipeline óptima fronte a un *stack* de técnicas estado da arte da literatura.
- OE7.** Materializar todo o anterior nun marco experimental reproducible e parametrizable (implementado con Kedro) que constitúa un entregable reutilizable, aplicable á optimización de sistemas RAG en calquera dominio documental.

O documento organízase do seguinte modo: a sección de Planificación describe o deseño metodolóxico e a organización temporal do traballo. O bloque de Desenvolvemento do traballo abarca o marco teórico (Sección 1), a descrición do corpus (Sección 2), a metodoloxía experimental (Sección 3) e os resultados e a súa discusión (Sección 4). Finalmente, as Conclusións e ampliación recollen os achados principais, as limitacións e as liñas de traballo futuro.

Planificación

O traballo organízase en seis fases secuenciais que reflicten directamente a estrutura do deseño experimental descrita na Sección 3.2. A secuencialidade non é arbitraria: cada fase depende dos resultados da anterior, polo que non é posible avanzar ás fases de técnicas avanzadas sen fixar previamente o embedding, o chunking e o retrieval base.

Fase 1 — Revisión bibliográfica e marco teórico. Revisión sistemática do estado da arte en arquitecturas RAG, modelos de embedding, estratexias de segmentación, técnicas de retrieval e frameworks de avaliación. Resultado: Sección 1 (Estado da arte e marco teórico) redactada.

Fase 2 — Construción do corpus e do golden dataset. Descarga e limpeza dos documentos EUR-Lex seleccionados como corpus de avaliación. Xeración dos pares pregunta/resposta-ideal mediante LLM e revisión manual. Resultado: Sección 2 (Corpus documental) redactada e golden dataset validado.

Fase 3 — Implementación da infraestrutura RAG. Desenvolvemento do código base da pipeline RAG (inxesta, indexación, retrieval, xeración e avaliación RAGAS) e validación sobre un subconxunto do golden dataset. Resultado: infraestrutura funcional e reproducible sobre a que se executan todos os experimentos.

Fase 4 — Experimentos 0 a 3. Execución secuencial dos experimentos de baseline (Exp. 0), comparativa de embeddings (Exp. 1), comparativa de chunking (Exp. 2) e comparativa de técnicas de retrieval (Exp. 3). En cada experimento rexístranse todos os scores e fíxase a mellor configuración para a fase seguinte.

Fase 5 — Experimentos 4 e 5. Avaliación das técnicas de expansión de consultas (HyDE, reescrita, multi-query, RAG-Fusion, step-back, descomposición, self-query) e do reranking pos-recuperación (cross-encoder e LLM listwise) sobre o mellor RAG base acumulado da Fase 4, e validación final contra un *stack* estado da arte.

Fase 6 — Análise de resultados e redacción. Análise comparativa de todos os experimentos, extracción de conclusións por pregunta de investigación e redacción final da Sección 3 (Metodoloxía experimental) e do capítulo de Conclusións.

Táboa 1: Planificación temporal do traballo

Fase	Período	Actividade
1	Febreiro 2026	Revisión bibliográfica e marco teórico
2	Febreiro–Marzo 2026	Corpus EUR-Lex e golden dataset
3	Marzo 2026	Implementación da infraestrutura RAG
4	Marzo–Abril 2026	Experimentos 0–3 (baseline, embeddings, chunking, retrieval)
5	Abril 2026	Experimentos 4 e 5 (expansión de consultas, reranking)
6	Abril–Maio 2026	Análise de resultados e redacción final

Fonte: elaboración propia

Desenvolvemento do traballo

1. Estado da arte e marco teórico

1.1. Modelos de linguaxe de gran escala

1.1.1. A arquitectura Transformer

O nacemento dos sistemas RAG está indisolublemente ligado ao desenvolvemento da arquitectura Transformer, proposta por Vaswani et al. (2017) en 2017. Con anterioridade, os modelos de linguaxe baseábanse en redes recorrentes (RNN, LSTM) que procesaban os tokens de forma secuencial, o que limitaba a paralelización e dificultaba a captura de dependencias a longa distancia. A contribución central de Vaswani et al. foi substituír a recurrencia polo mecanismo de *self-attention*, que permite ao modelo relacionar de forma directa calquera par de posicións nunha secuencia, independentemente da súa distancia.

O mecanismo de *self-attention* calcula, para cada token, un vector de representación contextualizado como suma ponderada dos valores de todos os tokens da secuencia. Formalmente, exprésase do seguinte xeito:

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right) V \quad (1)$$

onde $Q \in \mathbb{R}^{n \times d_k}$, $K \in \mathbb{R}^{m \times d_k}$ e $V \in \mathbb{R}^{m \times d_v}$ son as matrices de queries, keys e values respectivamente, e d_k é a dimensión das representacións. O factor de escala $1/\sqrt{d_k}$ evita que o produto interior crezca excesivamente cando d_k é grande, o que degradaría os gradientes do softmax.

Intuitivamente, cada token xera tres vectores mediante proxeccións lineais aprendidas a partir do seu embedding x_i : unha consulta ou *query* ($q_i = x_i W^Q$), que representa que información busca ese token; unha clave ou *key* ($k_i = x_i W^K$), que representa que información ofrece; e un valor ou *value* ($v_i = x_i W^V$), que representa o contido que se transmite se a busca é relevante. A puntuación de atención entre dous tokens i e j obtense do produto interior $q_i \cdot k_j$, que mide a compatibilidade entre o que un busca e o que o outro ofrece; tras normalizar estas puntuacións con softmax, empréganse como pesos para combinar os *values* de todos os tokens e producir a representación contextualizada de cada posición.

O cualificativo *multi-cabeza* indica que este proceso de proxección e atención se repite en paralelo h veces, cada unha cun conxunto independente de matrices W^Q, W^K, W^V aprendidas. Cada cabeza pode así especializarse en capturar un tipo distinto de relación entre tokens (por exemplo, dependencias sintácticas próximas ou correferencias a longa distancia). As saídas das h cabezas concaténanse e proxéctanse de novo para producir a representación final da capa.

Segundo a procedencia de Q, K e V , distínguense dous tipos de atención. En *self-attention* —a empregada na ecuación anterior—, as tres matrices derívanse da mesma secuencia de entrada, de forma que cada token atende a todos os tokens (incluído el mesmo) dentro dese mesmo contexto. En *cross-attention*, pola contra, Q procede dunha secuencia mentres que K e V proceden doutra

secuencia distinta: por exemplo, na arquitectura encoder-decoder orixinal, o decoder xera as queries e atende sobre as keys e values producidos polo encoder. Esta é a mecánica que emprega a proposta orixinal de RAG (Lewis et al., 2020) para condicionar a xeración sobre os documentos recuperados.

Esta arquitectura alcanzou resultados estado da arte en tradución automática, con 28.4 BLEU en English-to-German e 41.8 BLEU en English-to-French no benchmark WMT 2014 (Vaswani et al., 2017).

1.1.2. De BERT a GPT: dúas familias de LLM

A arquitectura Transformer deu lugar a dúas familias de modelos con obxectivos distintos, ambas de enorme relevancia para os sistemas RAG.

BERT (*Bidirectional Encoder Representations from Transformers*), proposto por Devlin et al. (2019), é un modelo baseado exclusivamente no encoder do Transformer. O seu preadestramento realízase mediante dúas tarefas: *Masked Language Modelling* (MLM), na que se enmascara o 15 % dos tokens e se solicita ao modelo que os prediga, e *Next Sentence Prediction* (NSP). Grazas ao procesamento bidireccional, BERT xera representacións profundamente contextualizadas especialmente útiles para tarefas de comprensión lectora, clasificación e recuperación de información. O modelo alcanzou melloras de 7.7 puntos absolutos no benchmark GLUE respecto ao estado da arte anterior (Devlin et al., 2019).

GPT (*Generative Pre-trained Transformer*), en cambio, emprega só o decoder, operando de forma autorregresiva: a probabilidade de cada token está condicionada exclusivamente polo contexto previo (causal attention). GPT-3 (Brown et al., 2020), con 175 mil millóns de parámetros, demostrou que a escala por si soa dota os modelos de capacidades emerxentes como a aprendizaxe few-shot e a resolución de razoamento analóxico sen requirir axuste fino específico por tarefa.

1.1.3. Limitacións dos LLM: alucinacións e coñecemento estático

A pesar do seu impresionante rendemento, os LLM presentan limitacións fundamentais:

Alucinación factual. Os LLM xeran texto con base en distribucións de probabilidade aprendidas dos datos de adestramento, non en razoamento sobre feitos verificados. Ji et al. (2023) realizaron unha revisión exhaustiva deste fenómeno, identificando as súas causas, métricas de medición e estratexias de mitigación.

Coñecemento estático. O coñecemento dun LLM está conxelado na data de corte do seu adestramento, o que limita gravemente a súa utilidade en dominios dinámicos como o xurídico, no que a lexislación evoluciona de forma continua.

Falta de trazabilidade. Os LLM non poden citar as fontes das que procede a información que xeran, o que dificulta a verificación en contextos de alta responsabilidade.

Limitación da xanela de contexto. Procesar corpus completos de miles de documentos en cada inferencia resulta computacionalmente prohibitivo e introduce ruído innecesario.

Os sistemas RAG abordan as tres primeiras limitacións de forma directa, ao proporcionar ao modelo fragmentos de texto verificables e trazables en tempo de inferencia.

1.2. Retrieval-Augmented Generation: fundamentos

1.2.1. A proposta orixinal de Lewis et al.

Lewis et al. (2020) introduciron o paradigma RAG como un método de axuste fino de propósito xeral para tarefas de NLP intensivas en coñecemento. O sistema consta de dous compoñentes principais: un recuperador (retriever) baseado en DPR (Karpukhin et al., 2020) e un xerador seq2seq baseado en BART. Dada unha consulta x , o recuperador obtén os k documentos máis relevantes dun índice vectorial de Wikipedia, e o xerador produce a resposta y condicionando sobre x e os documentos recuperados $z_1, \dots, z_k$.

Os autores propuxeron dúas variantes do modelo:

- **RAG-Sequence:** usa o mesmo documento z_i para xerar todos os tokens da resposta, e marginaliza sobre os k documentos ao nivel da secuencia completa.
- **RAG-Token:** pode usar documentos distintos para xerar diferentes tokens da resposta, o que ofrece maior flexibilidade pero é máis custoso computacionalmente.

O sistema estableceu o estado da arte en tres tarefas de *open-domain QA* —Natural Questions, WebQuestions e CuratedTrec— e, en *fact verification* (FEVER), situouse a menos dun 4,3 % do estado da arte, representado por sistemas de *pipeline* moito máis complexos que requiren supervisión explícita da recuperación (Lewis et al., 2020). Estes resultados consolidaron RAG como paradigma de referencia para a xeración baseada en coñecemento factual.

1.2.2. Taxonomía de sistemas RAG

A revisión de Y. Gao et al. (2023) sistematiza a evolución do paradigma RAG en tres xeracións:

Naive RAG. Representa a implementación máis directa: indexación dos documentos mediante embeddings, recuperación por similitude vectorial e xeración aumentada. A súa sinxeleza implica limitacións notables na calidade da recuperación e ausencia de mecanismos de filtrado ou reordenación.

Advanced RAG. Introduce melloras en cada fase da pipeline: segmentación semántica, estratexias híbridas de recuperación, reranking e expansión de consultas antes da xeración, e compresión e filtrado do contexto.

Modular RAG. Reconceptualiza o sistema como un conxunto de módulos intercambiabiles (retriever, reranker, compressor, xerador) que se poden combinar e reconfigurar de forma flexible, incorporando melloras como Self-RAG (Asai et al., 2024) ou a recuperación adaptativa.

1.2.3. Pipeline RAG estándar

Unha pipeline RAG estándar (*Advanced RAG*) compóñese das seguintes fases:

1. **Inxesta e segmentación.** Os documentos brutos son cargados, limpados e divididos en fragmentos (*chunks*) de tamaño manexable.

2. **Indexación.** Cada chunk é codificado mediante un modelo de embedding e os vectores almacénanse nun índice de busca de veciños aproximados (Approximate Nearest Neighbour (ANN)).
3. **Recuperación.** Ante unha consulta do usuario, esta codifícase co mesmo modelo de embedding e realízase unha busca no índice para recuperar os k chunks máis similares.
4. **Pos-recuperación (opcional).** Os chunks recuperados pódense reordenar, filtrar ou comprimir antes de ser entregados ao xerador.
5. **Xeración.** O modelo de linguaxe recibe a consulta do usuario e os chunks recuperados como contexto, e xera a resposta final.

1.3. Compoñentes dunha pipeline RAG

1.3.1. Segmentación de documentos (chunking)

A granularidade coa que se dividen os documentos para a indexación é un dos factores máis determinantes no rendemento dun sistema RAG (Shi et al., 2024). Chunks demasiado grandes incorporan información irrelevante que dilúe o sinal útil; chunks demasiado pequenos poden carecer de contexto suficiente para producir respostas coherentes. As principais estratexias son:

Chunking de tamaño fixo. Divide o texto en fragmentos de n tokens sen ter en conta a estrutura semántica do documento. É a aproximación máis simple e computacionalmente eficiente, pero pode cortar frases ou parágrafos á metade.

Chunking con solapamento. Variante do anterior na que fragmentos consecutivos comparten m tokens de solapamento (tipicamente $m \approx n/8$). O solapamento reduce o risco de perder información relevante que quede na fronteira entre dous chunks, á custa de aumentar o tamaño do índice.

Chunking recursivo. Emprega unha xerarquía de separadores (parágrafo, frase, palabra) para dividir o texto de forma adaptativa, respectando os límites semánticos naturais. É a estratexia por defecto en LangChain (`RecursiveCharacterTextSplitter`) e ofrece un bo equilibrio entre calidade e eficiencia.

Chunking semántico. Encoda cada frase como un embedding e segmenta o documento nas posicións onde a similitude coseno entre frases consecutivas cae por debaixo dun limiar (Kamradt, 2023). Produce chunks semanticamente coherentes, pero é computacionalmente máis custoso e require axuste do limiar de segmentación.

Chunking por cabeceira. Segmenta o documento usando os marcadores explícitos de estrutura: cabeceiras de sección en documentos HTML ou Markdown, ou os títulos de artigos en textos xurídicos. Os fragmentos resultantes corresponden a unidades semánticas naturais do corpus, o que os fai especialmente eficaces en documentos cun esquema formal predecible como os regulamentos e directivas de EUR-Lex (Y. Gao et al., 2023).

Chunking pai-fillo (*parent-child chunking*). Mantén dúas granularidades simultáneas: fragmentos pequenos (fillo) que se indexan no vector store para maximizar a precisión na recuperación, e fragmentos grandes (pai) que se entregan ao xerador para proporcionar contexto suficiente. Na busca, os chunks fillo actúan como sondas de recuperación, mais a resposta xérase a partir do seu fragmento pai correspondente (Chase, 2022).

Xanela deslizante (*sliding window*). Xera fragmentos de lonxitude fixa desprazándose polo texto cun paso menor que o tamaño do fragmento. A diferenza do chunking con solapamento, o paso e o tamaño son parámetros independentes, o que ofrece un control máis granular sobre o grao de redundancia entre chunks contiguos.

1.3.2. Modelos de embedding e almacenamento vectorial

Os modelos de embedding mapean texto de lonxitude variable nun vector denso de dimensión fixa de forma que textos semánticamente similares producen vectores próximos no espazo euclidiano. Sentence-BERT (Reimers e Gurevych, 2019) introduciu o uso de redes siamesas sobre BERT para producir embeddings de oración computacionalmente eficientes, reducindo o tempo de busca de pares máis similares de 65 horas a aproximadamente 5 segundos.

O almacenamento e a busca eficiente dos vectores realízase mediante índices de ANN. A estratexia máis empregada na práctica é HNSW (*Hierarchical Navigable Small World*) (Malkov e Yashunin, 2020): un grafo de veciños organizado en capas que permite navegar rapidamente cara aos vectores máis próximos, ofrecendo un compromiso eficiente entre velocidade de busca e precisión. Outras estratexias como IVF (*Inverted File Index*, que particiona o espazo en clusters) ou PQ (*Product Quantization*, que comprime os vectores en códigos curtos) reducen aínda máis o custo de memoria á custa de precisión. Neste traballo emprégase Qdrant (Qdrant, 2024), unha base de datos vectorial de código aberto que implementa HNSW de forma nativa para a indexación.

Nos últimos anos xurdiron diversas familias de modelos de embedding especializadas para recuperación de información. A familia BAAI General Embedding (BGE) (BAAI *General Embedding*, Xiao et al. 2023) ofrece variantes de tamaño reducido (bge-small-en-v1.5), intermedio (bge-base-en-v1.5) e grande (bge-large-en-v1.5), optimizadas para busca de pasaxes en inglés mediante preadestramento con pares de oracións positivos e negativos. Para corpus con texto en varias linguas europeas, os modelos da familia **multilingual-e5** (Wang et al., 2022) (multilingual-e5-base e multilingual-e5-large) e paraphrase-multilingual-mpnet-base-v2 (Reimers e Gurevych, 2019) ofrecen representacións densas que capturan similitudes semánticas entre distintas linguas oficiais da UE, o que os fai especialmente adecuados para corpus como EUR-Lex.

1.3.3. Recuperación: densa, esparsa e híbrida

Existen dous paradigmas clásicos de recuperación de información:

Recuperación esparsa (BM25). BM25 (Robertson e Zaragoza, 2009) é a función de ranqueo baseada en TF-IDF máis amplamente usada en sistemas de recuperación de información. Non require aprendizaxe e é robusta para consultas con terminoloxía específica. Para dominios especializados como o xurídico, onde os termos técnicos son altamente discriminativos, BM25 pode

superar os sistemas de recuperación densa en ausencia de corpus de adestramento do dominio.

Recuperación densa (DPR). Karpukhin et al. (2020) propuxeron o uso dun encoder dual (bi-encoder) baseado en BERT para producir embeddings de consulta e documento no mesmo espazo vectorial, permitindo calcular a súa relevancia mediante o produto interior.

Recuperación híbrida. Combina as puntuacións de ambos sistemas mediante fusión por rango (RRF, Reciprocal Rank Fusion) ou combinación lineal ponderada, buscando aproveitar a complementariedade entre a correspondencia léxica de BM25 e a comprensión semántica da recuperación densa.

1.4. Técnicas avanzadas de optimización

1.4.1. Reranking

Os modelos bi-encoder usados na recuperación densa sacrifican precisión en favor da eficiencia: codifican consulta e documento de forma independente. Os modelos *cross-encoder* resólvén este problema procesando o par (consulta, documento) de forma conxunta, producindo unha puntuación de relevancia significativamente máis precisa (Nogueira e Cho, 2019).

O proceso de reranking consiste en aplicar un cross-encoder aos k documentos recuperados polo bi-encoder e reordenalos segundo a nova puntuación, retendo só os $k' \leq k$ mellores. Este proceso é computacionalmente viable porque opera sobre un conxunto pequeno de documentos preseleccionados.

MMR (*Maximal Marginal Relevance*) é outra técnica de reordenación que non maximiza só a relevancia dos documentos recuperados senón tamén a diversidade entre eles, evitando que os top- k resultados sexan altamente redundantes entre si.

1.4.2. Expansión de consultas: HyDE

Un dos problemas da recuperación densa é a asimetría semántica entre consultas e documentos: as consultas son tipicamente curtas e informais, mentres que os documentos son longos e formais.

L. Gao et al. (2023) propuxeron HyDE (*Hypothetical Document Embeddings*), un enfoque que resolve esta asimetría en cero disparos (*zero-shot*): dada unha consulta, un LLM xera un documento hipotético que respondería á consulta. A continuación, ese documento hipotético (non a consulta orixinal) úsase como vector de busca no índice. O documento hipotético, aínda que sexa factualmente incorrecto, captura mellor os patróns de relevancia dun documento real que a consulta orixinal.

1.4.3. Reescrita de consultas e recuperación multi-consulta

A asimetría léxica entre as consultas de usuario (tipicamente curtas e coloquiais) e o texto dos documentos do corpus (formal e técnico) constitúe un dos factores de degradación máis frecuentes nos sistemas RAG sobre dominios especializados. Dous enfoques complementarios abordan este problema:

Reescrita de consultas (*query rewriting*): un LLM transforma a consulta orixinal nunha versión reescrita con vocabulario máis próximo ao do corpus. Esta versión reescrita emprégase

na busca vectorial en lugar da consulta orixinal, reducindo a asimetría léxica sen necesitar un documento hipotético completo como en HyDE.

Recuperación multi-consulta (*multi-query retrieval*): o LLM xera n variantes da consulta orixinal, realízase a busca por cada variante de forma independente, e os resultados agréganse mediante unión ou fusión por rango. Isto reduce o risco de perder chunks relevantes que só son recuperables con determinadas formulacións da mesma pregunta.

1.4.4. Self-query retriever

O *self-query retriever* (Chase, 2022) usa un LLM para descompoñer a consulta do usuario en dúas partes: (1) un conxunto de filtros de metadatos estruturados (por exemplo, `tipo=regulamento` ou `ano>2020`) e (2) unha consulta semántica residual sobre o contido. A busca execútase aplicando primeiro os filtros sobre o vector store e, a continuación, a recuperación densa sobre o subconxunto filtrado. Esta técnica é especialmente valiosa en corpus como EUR-Lex, onde os documentos están anotados con metadatos ricos (tipo de acto, número CELEX, data de publicación).

1.4.5. Recuperador pai-fillo (*parent document retriever*)

O recuperador pai-fillo materializa a estratexia de chunking pai-fillo na pipeline RAG completa (Chase, 2022). O índice vectorial contén os fragmentos fillo, de tamaño reducido (e.g., 256 tokens), o que favorece alta precisión na recuperación. Non obstante, ao modelo xerador non se lle entrega o fragmento fillo recuperado senón o seu fragmento pai (e.g., 1024 tokens), que contén o fragmento fillo máis o contexto circundante. Desta forma combínase a precisión do retriever coa riqueza de contexto necesaria para a xeración.

1.4.6. Outras técnicas do estado da arte

As técnicas avaliadas nas seccións anteriores —reranking, expansión e transformación de consultas, self-query e recuperación pai-fillo— constitúen xa aproximacións do estado da arte, e son as que se integran na comparativa empírica deste traballo. Xunto a elas, a literatura recente propón outras estratexias igualmente avanzadas que, por limitacións de alcance, quedan fóra da avaliación empírica. No plano dos compoñentes da pipeline, a **compresión de contexto** filtra ou resume os documentos recuperados antes de entregalos ao xerador, e a **recuperación contextual** antepón a cada chunk un breve resumo do seu papel no documento para mellorar a súa recuperabilidade. No plano arquitectónico, diversos enfoques dotan o sistema de capacidade de decisión sobre o propio proceso de recuperación: **Self-RAG** (Asai et al., 2024) introduce *reflection tokens* que deciden cando recuperar e se a resposta está fundamentada; **CRAG** (Yan et al., 2024) engade un módulo de corrección que dispara buscas web cando os chunks recuperados son insuficientes; **Adaptive-RAG** (Jeong et al., 2024) clasifica cada consulta pola súa complexidade para axustar o esforzo de recuperación; o **RAG axéntico** con LangGraph (Chase, 2022) reconceptualiza a pipeline como un axente con planificación e control de fluxo; e **GraphRAG** (Edge et al., 2024) substitúe a indexación de chunks illados por un grafo de coñecemento que permite razoar sobre relacións dispersas no corpus. Varias destas arquitecturas retómanse como liñas de traballo futuro.

1.5. Frameworks de avaliación para RAG

A avaliación de sistemas RAG presenta un reto particular: ademais da calidade da recuperación (¿son relevantes os chunks recuperados?), hai que avaliar a calidade da xeración (¿a resposta é correcta, está fundamentada no contexto, responde realmente á pregunta?), aspectos que as métricas clásicas de recuperación de información (precisión, recall, etc.) non capturan, xa que requiren xulgar a corrección semántica dun texto en linguaxe natural fronte a un conxunto de referencias. O paradigma *LLM-as-judge* (LLM como xuíz) aborda este problema empregando un modelo de linguaxe de gran escala para emitir xuízos automáticos sobre estes aspectos —por exemplo, decidir se unha afirmación está apoiada por un fragmento de texto, ou xerar preguntas hipotéticas a partir dunha resposta para comparalas coa pregunta orixinal—, substituíndo así anotacións manuais por chamadas a un LLM con prompts deseñados especificamente para cada métrica. Isto permite avaliar sistemas RAG sen necesidade de anotación humana exhaustiva (*reference-free*), aínda que introduce unha dependencia da calidade e dos sesgos do modelo avaliador.

1.5.1. RAGAS

RAGAS (*Retrieval Augmented Generation Assessment*, Es et al. 2023) é o framework de avaliación de referencia para sistemas RAG. Non require anotacións humanas de referencia (*reference-free*): emprega un LLM como xuíz para calcular as métricas a partir da consulta do usuario (q), o contexto recuperado (c), a resposta xerada (a) e, opcionalmente, a resposta de referencia (g). As métricas principais son:

Faithfulness. Mide en que medida as afirmacións contidas na resposta xerada están apoiadas polo contexto recuperado. Formalmente:

$$\text{Faithfulness} = \frac{|\{s \in a : s \text{ apoiado en } c\}|}{|a|} \quad (2)$$

Answer Relevancy. Mide se a resposta xerada aborda a pregunta do usuario. Calcúlase xerando consultas hipotéticas a partir da resposta e medindo a similitude coseno entre as consultas hipotéticas e a consulta orixinal. Formalmente:

$$\text{AnswerRelevancy} = \frac{1}{n} \sum_{i=1}^n \cos(E(q), E(q'_i)) \quad (3)$$

onde q'_i ($i = 1, \dots, n$) son n preguntas hipotéticas xeradas por un LLM a partir da resposta a , q é a pregunta orixinal e $E(\cdot)$ denota o embedding dun texto.

Context Precision. Mide se os documentos relevantes recuperados están posicionados nos rangos superiores do contexto. Formalmente, para os K chunks recuperados:

$$\text{ContextPrecision@K} = \frac{\sum_{k=1}^K (\text{Precision@k} \times v_k)}{\sum_{k=1}^K v_k} \quad (4)$$

onde $v_k \in \{0, 1\}$ indica se o chunk na posición k é relevante para responder á pregunta (xulgado por un LLM respecto á resposta de referencia) e Precision@k é a proporción de

chunks relevantes entre os k primeiros.

Context Recall. Mide en que medida o contexto recuperado contén toda a información necesaria para responder á pregunta, comparando co ground truth. Formalmente:

$$\text{ContextRecall} = \frac{|\{s \in g : s \text{ atribuíble a } c\}|}{|g|} \quad (5)$$

onde g é o conxunto de afirmacións da resposta de referencia do golden dataset e c o contexto recuperado.

Context Entity Recall. Mide a proporción de entidades presentes na resposta de referencia do golden dataset que tamén aparecen no contexto recuperado. Complementa a *Context Recall* centrándose nas entidades (nomes propios, referencias normativas, datas, organismos), especialmente relevantes nun dominio xurídico onde a identificación correcta de directivas, regulamentos e asuntos é crítica. Formalmente:

$$\text{ContextEntityRecall} = \frac{|GE \cap CE|}{|GE|} \quad (6)$$

onde GE é o conxunto de entidades extraídas da resposta de referencia g e CE o conxunto de entidades extraídas do contexto recuperado c , ambas identificadas por un LLM.

Noise Sensitivity. Mide a proporción de afirmacións incorrectas na resposta atribuíbles a chunks irrelevantes presentes no contexto recuperado. Cuantifica o impacto negativo do ruído de recuperación sobre a calidade da xeración, e é especialmente relevante na comparativa de técnicas de retrieval. Formalmente:

$$\text{NoiseSensitivity} = \frac{|\{s \in a : s \text{ non apoiada por } g \wedge s \text{ atribuíble a chunks irrelevantes de } c\}|}{|a|} \quad (7)$$

onde unha afirmación se considera *incorrecta* cando non está apoiada pola resposta de referencia g .

1.5.2. Mecánica do LLM-as-judge e determinismo

As fórmulas anteriores expresan *que* mide cada métrica, pero non *como* se obteñen os xuízos. RAGAS non lle pide ao LLM unha puntuación holística —inestable e difícil de auditar—, senón que descompón cada avaliación nun conxunto de decisións binarias atómicas sobre as que despois opera aritmeticamente:

- **Descomposición en afirmacións.** Para as métricas baseadas na resposta (faithfulness, context recall, noise sensitivity), un primeiro paso pídelles ao LLM que descompoña o texto (a resposta xerada ou a de referencia) nun conxunto de afirmacións simples e autocontidas, sen pronomes.
- **Veredicto binario tipo NLI.** Para cada afirmación, o LLM emite un veredicto 0/1 resolvendo unha tarefa de inferencia textual: por exemplo, ¿pode inferirse esta afirmación a partir do contexto recuperado? (faithfulness) ou ¿é atribuíble esta afirmación da resposta de referencia ao contexto? (context recall). A puntuación é a proporción de veredictos positivos.

- **Saída estruturada.** Cada veredicto prodúcese en JSON cun esquema fixo (afirmación, razón, veredicto) guiado con exemplos *few-shot*, o que forza respostas parseables e reduce a variabilidade fronte á xeración en texto libre.

Non todas as métricas se reducen a veredictos do LLM. *Answer relevancy* combina o LLM (que xera tres preguntas hipotéticas a partir da resposta e detecta se é evasiva) cun modelo de **embeddings**, que mide a similitude coseno entre esas preguntas e a orixinal; *context entity recall* apóiase nun paso de extracción de entidades por LLM; e *context precision* xulga, para cada chunk recuperado e en orde de rango, se foi útil para chegar á resposta de referencia, ponderando o resultado pola posición (precisión media) de modo que premia que os chunks relevantes aparezan arriba.

Como un LLM é intrinsecamente estocástico, RAGAS adopta varias medidas para que a avaliación sexa o máis reproducible posible. As chamadas de xuízo execútanse cunha temperatura próxima a cero ($\approx 0,01$); cando unha métrica mostrea varias xeracións para unha mesma decisión, RAGAS sobe lixeiramente a temperatura (0,3) para obter diversidade e agrega os veredictos por **vo-to maioritario**. A combinación de temperatura baixa, saída estruturada e descomposición en moitas decisións binarias pequenas —en lugar dunha única puntuación global— reduce substancialmente a varianza entre execucións. O modelo xuíz empregado é *gemini-2.5-flash* (temperatura 0 e razoamento interno desactivado), e os embeddings requiridos por *answer relevancy* calcúlanse co modelo *gemini-embedding-2*. Cómpre subliñar que o xuíz pertence a unha familia de modelos distinta á do xerador das respostas e á do modelo que construíu o golden dataset, ambos *deepseek-v4-flash*.

1.5.3. Diferenzas coa avaliación clásica e limitacións

A avaliación automática de sistemas de xeración apoiouse tradicionalmente en métricas léxicas baseadas no solapamento de n-gramas coa resposta de referencia, como BLEU ou ROUGE. Son deterministas e baratas, pero penalizan respostas correctas formuladas con palabras distintas á referencia e non captan a equivalencia semántica nin a fidelidade ao contexto, dúas dimensións centrais nun sistema RAG. A alternativa clásica —a anotación humana— é fiable pero lenta e custosa, e non escala aos centenaes de configuracións que require unha ablación coma a deste traballo. O paradigma LLM-as-judge sitúase no medio: aproxima o xuízo semántico humano a un custo e velocidade compatibles cunha avaliación masiva. Esta capacidade ten, no entanto, limitacións que deben terse en conta:

- **Dependencia do modelo avaliador.** As puntuacións están condicionadas pola competencia e os nesgos do LLM xuíz; outro modelo podería emitir veredictos distintos sobre os mesmos exemplos.
- **Nesgo de auto-preferencia.** Cando o mesmo modelo xera e avalía contido, tende a puntuar máis alto o texto semellante ao seu propio estilo. Neste traballo este risco evítase por deseño: o modelo xuíz (*gemini-2.5-flash*) pertence a unha familia distinta á do modelo que xera as respostas e constrúe o golden dataset (*deepseek-v4-flash*), de modo que o avaliador nunca xulga a súa propia saída. Ademais, os compoñentes comparados (embeddings, chunking, etc.) son independentes do xuíz e idénticos salvo na variable en estudo, polo

que calquera nesgo residual do avaliador afecta por igual a todas as configuracións dunha mesma comparación.

- **Outros nesgos coñecidos.** A literatura documenta no LLM-as-judge nesgos de verbosidade (preferencia por respostas máis longas), de posición (sensibilidade á orde dos elementos a xulgar) e unha alta sensibilidade á redacción do prompt.
- **Determinismo imperfecto e custo.** A pesar das medidas anteriores, persiste certa varianza entre execucións, e cada métrica require varias chamadas ao LLM por exemplo, o que impón un custo económico e limita o número de configuracións avaiables.

Por estas razóns, as métricas RAGAS interprétanse neste traballo como **indicadores comparativos** entre configuracións avaliadas nas mesmas condicións, e non como medidas absolutas de calidade.

1.6. Framework de pipelines: Kedro

Kedro (QuantumBlack, AI by McKinsey, 2023) é un framework de código aberto para a construción de pipelines de datos e ML reproducibles e modulares. Organiza o código en *nodos* (funcións puras sen efectos secundarios) e *pipelines* declarativos, e xestiona as dependencias entre etapas e os artefactos intermedios mediante un catálogo de datos centralizado. Deste xeito, cada execución é reproducible e auditable sen requirir xestión manual de rutas de ficheiros ou scripts de orquestración ad hoc.

No presente traballo, Kedro estrutura a totalidade da infraestrutura experimental, e non só unha etapa illada. O sistema organízase en pipelines independentes —selección do corpus (`corpus_selection`), inxesta e limpeza (`ingestion`), construción do golden dataset (`gold_dataset`), execución RAG (`rag_base`) e avaliación RAGAS (`evaluation`)— e cada experimento da ablación defínese mediante un ficheiro de parámetros propio (`parameters_exp*.yaml`), sen modificar o código. Esta separación entre a lóxica de procesamento e a configuración é o que fai que cada experimento sexa reproducible e auditable, e que substituír o corpus, o modelo de embedding ou calquera outro compoñente se reduza a cambiar parámetros. Esta arquitectura é a que sustenta a contribución instrumental do traballo discutida nas conclusións: o marco non está acoplado ao caso de EUR-Lex, senón que é directamente reutilizable noutros dominios documentais.

1.7. Revisión de traballos relacionados

Varios traballos recentes avalían empiricamente técnicas de optimización RAG, aínda que con alcances e corpus distintos ao presente estudo. Shi et al. (2024) realizaron unha análise sistemática de estratexias de chunking para QA, concluíndo que o chunking recursivo ofrece resultados competitivos con menor custo computacional que o chunking semántico. O traballo de Es et al. valida RAGAS como framework de avaliación comparándoo con anotacións humanas, obtendo alta correlación. Asai et al. (2024) demostran que a recuperación adaptativa de Self-RAG supera consistentemente a recuperación non selectiva en seis tarefas distintas. Yan et al. (2024) propoñen CRAG como mecanismo de corrección activa cando o retriever falla, e Jeong et al. (2024)

demostran que clasificar as consultas por complexidade antes de decidir a estratexia de retrieval reduce a latencia sen sacrificar calidade. Edge et al. (2024) propoñen GraphRAG como alternativa á recuperación vectorial estándar para preguntas que requiren síntese de información dispersa.

O presente traballo diferénciase dos anteriores en tres aspectos: (1) aplica a avaliación a un dominio xurídico especializado (EUR-Lex) con alta densidade terminolóxica; (2) combina nunha única infraestrutura experimental, con deseño secuencial e base fija en cada fase, todas as técnicas mencionadas para permitir comparacións directas e controladas; e (3) incorpora seis métricas RAGAS que cobren tanto a calidade da recuperación como a da xeración de cada configuración.

2. Corpus documental

2.1. EUR-Lex como fonte de datos

EUR-Lex é o repositorio dixital oficial da lexislación da Unión Europea, xestionado pola Oficina de Publicacións da UE (Publications Office of the European Union, 2024). Ofrece acceso libre e gratuíto a máis de tres millóns de actos xurídicos en todas as linguas oficiais da UE, incluíndo regulamentos, directivas, decisións, xurisprudencia do Tribunal de Xustiza e acordos internacionais.

O corpus EUR-Lex presenta varias características que o fan especialmente adecuado para avaliar sistemas RAG en dominios especializados:

- **Alta densidade terminolóxica.** A lexislación europea emprega un vocabulario altamente especializado e preciso que non está ben representado nos datos de preadestramento xeral dos LLM.
- **Estrutura formal e predecible.** Os actos xurídicos da UE seguen un formato estandarizado (considerandos, artigos, anexos).
- **Disponibilidade e trazabilidade.** Todos os documentos están identificados por un identificador único CELEX e son accesibles de forma pública e reproducible.
- **Actualidade.** EUR-Lex inclúe lexislación actualizada até a data de descarga, permitindo avaliar a capacidade do sistema para proporcionar respostas baseadas en normativa recente.

2.2. Criterios de selección e características do corpus

O corpus baséase no conxunto de datos público `gplsi/alia_intellectual_property` (Espinosa Zaragoza et al., 2025) (CC BY 4.0), un subconxunto do repositorio EUR-Lex que os seus autores filtraron previamente para illar os documentos do dominio da propiedade intelectual. Deste dataset, o presente traballo emprega directamente a variante en inglés en formato Markdown (`eurlex-en-md.jsonl`) tal e como a publican os autores, sen aplicar ningunha filtraxe de dominio adicional neste nivel.

O criterio exacto co que os autores illaron o subconxunto de propiedade intelectual dentro do repositorio EUR-Lex completo non está documentado publicamente, polo que non é posible cuantificar a redución de tamaño que supuxo respecto ao repositorio orixe. A única selección

propia que realiza este traballo aplícase un nivel máis abaixo —para acoutar o dominio do dereito de autor (*copyright*)— e descríbese na Sección 2.4.

Este conxunto base —en diante, o **corpus fonte**— comprende **40.181 documentos** publicados entre 1957 e 2025, con cobertura de 207 tipos de actos distintos. Os documentos agrúpanse en catro familias xurídicas segundo a natureza do acto (Táboa 2). A lonxitude é moi heteroxénea: a mediana global é de 606 palabras, mentres que a media alcanza as 9.980 palabras debido á cola longa de documentos legislativos e de política de gran extensión.

Táboa 2: Composición do corpus EUR-Lex por familia de documentos

Familia	Tipos principais	N.º docs	%	Mediana (pal.)
JUDICIAL	Decisión xudicial, Sentenza, Orde	20.253	50,4	336
POLICY	Comunicación, Recomendación, Resolución	8.005	19,9	6.132
LEGISLATIVE	Regulamento, Directiva, Decisión	7.246	18,0	938
OTHER	Acordo internacional, Proposta, Aviso	4.677	11,6	1.588
Total	—	40.181	100	606

Fonte: elaboración propia a partir de datos de EUR-Lex

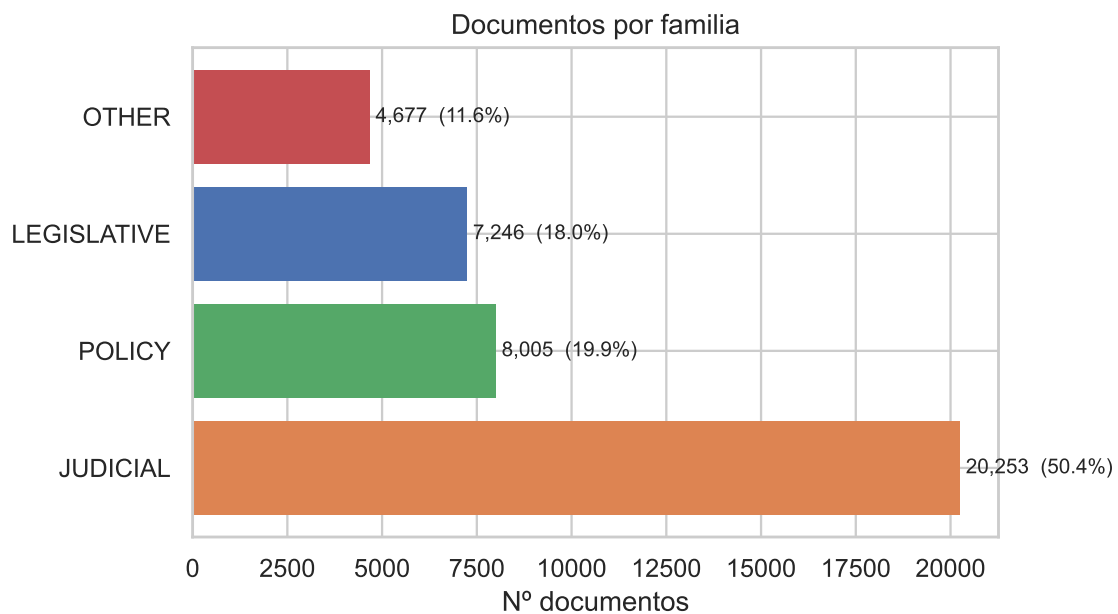


Figura 1: Distribución do corpus EUR-Lex por familia de documentos

Fonte: elaboración propia a partir de datos de EUR-Lex

A heteroxeneidade de lonxitude entre familias é moi marcada (Táboa 3 e Figura 2). A familia JUDICIAL, maioritaria no corpus, contén documentos curtos (mediana 336 palabras), cunha escisión natural entre avisos procesais (11.517 documentos, mediana 264 palabras) e xurisprudencia substantiva (8.736 documentos, mediana 631 palabras). A familia POLICY, pola contra, presenta os documentos máis extensos (mediana 6.132 palabras; percentil 90 de 56.880 palabras). Esta variabilidade, de tres ordes de magnitude entre familias, condiciona directamente as estratexias

de segmentación avaliadas no Experimento 2.

Táboa 3: Percentís de lonxitude por familia de documentos (palabras)

Familia	p25	p50 (mediana)	p75	p90
JUDICIAL	245	336	606	5.629
LEGISLATIVE	353	938	4.945	18.369
OTHER	377	1.588	8.050	24.733
POLICY	1.904	6.132	20.892	56.880

Mediana exacta (reconto de palabras); p25, p75 e p90 calculados como caracteres ÷ 5 (aproximación).

Fonte: elaboración propia a partir de datos de EUR-Lex

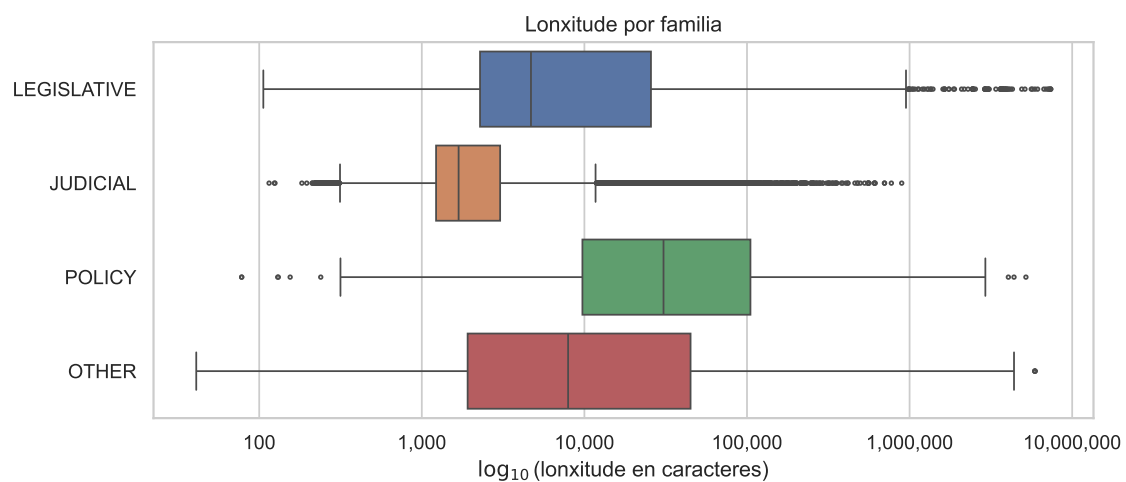


Figura 2: Distribución da lonxitude dos documentos por familia (escala logarítmica)

Fonte: elaboración propia a partir de datos de EUR-Lex

2.3. Preprocesamento e limpeza

Os documentos obtivéronse en formato JSONL desde o repositorio público `gplsi/alia_intellectual_property` (Espinosa Zaragoza et al., 2025) (`eurlex-en-md.jsonl`). Cada rexistro contén o texto do documento en formato Markdown (campo `content`) e un obxecto de metadatos estruturado (campo `metadata`) cos campos `celex`, `form`, `title`, `date`, `author` e `pages`, presentes no 100 % dos documentos do corpus.

A pipeline de inxesta (implementada como pipeline Kedro `ingestion`; véxase a Sección 1.6) aplica sobre o corpus de partida (`eurlex-en-md.jsonl`) a seguinte limpeza do texto:

1. `_strip_header`: Elimina a táboa de cabeceira da publicación (liñas do tipo `|`, `#`, `[`, `*`, `-`, `!`) ata o primeiro contido textual real (máis de 15 caracteres).
2. `_remove_footnote_anchors`: Suprime referencias de nota ao pé do tipo `[(1)(#ntr...)]` e similares.
3. `_remove_top_link`: Elimina a ligazón de navegación `[Top]` (`#document1`) e o separador `--` orfo que o precede.

4. `_strip_links`: `! [alt] (url) → nada`; `ligazóns [texto] (url) → só o texto`.
5. `_strip_markup`: Converte a texto plano: negritas `** . . . **`, cursivas `* . . *`, código en liña e cabeceiras `#`.
6. `_flatten_two_col_tables`: Táboas de dúas columnas `| (n) | texto |` (considerandos, subapartados de artigos) $\rightarrow$ texto plano continuo.
7. `_normalize_whitespace`: `\xa0` $\rightarrow$ espazo; 3 ou máis liñas en branco $\rightarrow$ 2; separadores `--` consecutivos $\rightarrow$ `un`.

O texto resultante conserva todo o contido normativo (artigos, considerandos, anexos, títulos en maiúsculas) e descarta exclusivamente os artefactos de formato introducidos polo sistema de publicación. A pipeline produce `corpus_clean.jsonl` cos 40.181 documentos do corpus fonte, listos para a selección do corpus de traballo descrita na Sección 2.4.

Esta etapa de limpeza implementouse integramente coa biblioteca estándar de Python (`re` e `json`), sen dependencias de procesamento de linguaxe natural.

2.4. Selección do corpus de traballo

O corpus fonte de 40.181 documentos abrangue todo o ámbito da propiedade intelectual, un espazo demasiado amplo e heteroxéneo para construír un conxunto de avaliación coherente con respostas verificables. Por iso, sobre o corpus xa limpo aplícase unha **selección propia** (pipeline `corpus_selection`) que illa un dominio temático máis acoutado —o dereito de autor (*copyright*) e dereitos afíns— co que construír o golden dataset e executar todos os experimentos. Un documento incorpórase ao corpus de traballo se cumpre as tres condicións seguintes:

1. O seu título contén algunha das palabras clave do dominio: *copyright*, *related right*, *neighbouring right*, *computer program* ou *software*.
2. A súa forma documental pertence a unha lista de tipos relevantes (sentenzas, conclusións do Avogado Xeral, autos, directivas, decisións, regulamentos, comunicacións, propostas e ditames, entre outras).
3. A súa lonxitude supera un limiar mínimo, para descartar documentos demasiado curtos para soste preguntas substantivas.

O resultado é un corpus de traballo de **363 documentos** (artefacto `corpus_copyright`). A súa composición por familia (Táboa 4) está aínda máis dominada pola xurisprudencia que o corpus fonte: a familia JUDICIAL representa o 70,5 % dos documentos, fronte ao 50,4 % no corpus completo, o que reflicte o peso da litixiosidade en materia de dereito de autor. Todos os resultados presentados na Sección 4 obtéñense sobre este corpus de traballo.

Táboa 4: Composición do corpus de traballo (subconxunto de copyright) por familia

Familia	N.º docs	%
JUDICIAL	256	70,5
POLICY	62	17,1
LEGISLATIVE	24	6,6
OTHER	21	5,8
Total	363	100

Fonte: elaboración propia a partir de datos de EUR-Lex

2.5. Construción do golden dataset de avaliación

A avaliación cuantitativa de calquera sistema RAG require un conxunto de probas con respostas de referencia verificables (*golden dataset*) que habilite métricas como *context recall* ou *faithfulness*. Dado o custo prohibitivo de redactar manualmente centenaes de pares pregunta-resposta sobre textos xurídicos, o conxunto xérase de forma sintética e semi-automática co módulo TestsetGenerator de RAGAS (v0.3.9) sobre o corpus de traballo de 363 documentos, seguindo catro etapas.

1. Mostraxe estratificada. Para evitar a sobrerepresentación dos documentos de lonxitude media, selecciónanse 75 documentos mediante mostraxe estratificada por lonxitude (10, 25, 25 e 15 documentos nos estratos de 300–1.000, 1.000–5.000, 5.000–20.000 e máis de 20.000 palabras, respectivamente). Os documentos longos sobrerepresentáanse deliberadamente (un 20 % da mostra fronte ao 9 % do corpus) por seren os máis ricos para preguntas que esixen combinar información. A selección dentro de cada estrato realízase cun xerador pseudoaleatorio inicializado cunha semente fixa (*seed*=42), o que fai a mostraxe reproducible.

2. Construción do grafo de coñecemento. RAGAS transforma os 75 documentos nun grafo de coñecemento: divídeos en fragmentos polos seus titulares, e sobre cada fragmento un LLM extrae os temas e as entidades nomeadas (organismos, referencias normativas, datas) e descarta os fragmentos de baixa calidade. A partir do solapamento de entidades entre fragmentos constrúense as relacións do grafo, que son as que posibilitan máis adiante as preguntas multi-salto.

3. Xeración de pares pregunta-resposta. Sobre o grafo xéranse escenarios e, a partir de cada un, un LLM produce un par (pregunta, resposta de referencia). A distribución é do 50 % entre dous tipos: *single-hop* (preguntas respondibles desde un único fragmento) e *multi-hop específico* (preguntas que esixen combinar feitos de dous fragmentos relacionados por entidades comúns). Cada pregunta condiciónase a unha de dúas *personas* —un avogado de propiedade intelectual e un analista de políticas— e a unha combinación de estilo (gramática correcta, gramática pobre, con erros ortográficos ou estilo de busca web) e lonxitude (curta, media ou longa), co fin de cubrir formulacións heteroxéneas semellantes ás dun usuario real. A resposta de referencia xérase coa instrución explícita de ser fiel exclusivamente ao contido dos fragmentos de orixe, o que a habilita como *ground truth*.

4. Filtrado e revisión. Os pares cuxa xeración produciu campos baleiros descártanse automaticamente; sobre o resto realízase unha revisión manual de coherencia para eliminar preguntas mal formadas ou ambiguas, sen reanotar as respostas. O resultado final é un conxunto de **149 pares** pregunta-resposta (75 *single-hop* e 74 *multi-hop*), cada un coa súa resposta de referencia e os fragmentos de contexto que a sustentan.

A xeración realízase co LLM `deepseek-v4-flash` (temperatura 0) e os embeddings do grafo de coñecemento co modelo `gemini-embedding-2`. Unha vez xerado, o golden dataset permanece fixo en todos os experimentos para garantir comparacións homoxéneas.

3. Metodoloxía experimental

3.1. Preguntas de investigación

Este traballo está guiado por cinco preguntas de investigación que se corresponden coa secuencia dos experimentos realizados:

RQ1: Cal é o modelo de embedding que mellor representa o corpus xurídico EUR-Lex en termos de *context recall* e *context precision*?

RQ2: Que estratexia de segmentación de documentos produce os chunks que mellor conservan o contexto relevante para as consultas do golden dataset?

RQ3: A recuperación híbrida (BM25 + vectorial), a diversificación Maximal Marginal Relevance (MMR) e a variación do número de documentos recuperados (k) melloran de forma consistente a recuperación densa estándar neste dominio?

RQ4: As técnicas que actúan sobre a consulta (reescrita, HyDE, multi-query, RAG-Fusion, step-back, descomposición, self-query) ou engaden un paso de reordenación pos-recuperación (*reranking* con cross-encoder ou LLM listwise) producen melloras relevantes sobre o mellor RAG base acumulado?

RQ5: A pipeline RAG construída empiricamente mediante a ablación secuencial supera en calidade a un *stack* que combina directamente técnicas consideradas estado da arte na literatura?

3.2. Deseño experimental

O deseño experimental segue un enfoque de *ablación secuencial* en cinco fases de optimización (embedding, chunking, retrieval, expansión de consultas e reranking), precedidas por unha liña base e seguidas dunha fase final de validación contra un *stack* estado da arte. En cada fase optimízase un compoñente da pipeline mentres o resto permanece fixo; a mellor configuración de cada fase convértese en constante para as fases seguintes. Este deseño garante que as comparacións dentro de cada experimento son válidas (varía só un compoñente á vez) e que o sistema RAG final está construído sobre as mellores eleccións en cada capa.

Esta estratexia constitúe unha optimización *greedy* (avara) e non garante un óptimo global: a configuración escollida en cada fase é a mellor condicionada ás decisións xa tomadas nas fases

anteriores, pero poderían existir interaccións entre compoñentes —por exemplo, entre o modelo de embedding e a estratexia de chunking— que fixesen que outra combinación, non explorada, superase a obtida secuencialmente. Esta limitación é análoga á da optimización secuencial de hiperparámetros, unha práctica estándar cando o espazo combinatorio completo (o produto cartesiano de todas as configuracións posibles en cada fase) resulta computacionalmente inabordable, e asume implicitamente que os efectos principais de cada compoñente predominan sobre as súas interaccións.

A secuencia de decisión é a seguinte:

1. **Experimento 0 (Baseline):** establece a referencia mínima honesta contra a que se miden todas as melloras.
2. **Experimento 1 (Embeddings):** fixa o mellor modelo de embedding para a recuperación en EUR-Lex.
3. **Experimento 2 (Chunking):** fixa a mellor estratexia de segmentación co embedding xa fixado.
4. **Experimento 3 (Retrieval):** fixa a mellor técnica de recuperación e o valor de k , co embedding e o chunking xa fixados.
5. **Experimento 4 (Expansión de consultas):** avalía sobre o mellor RAG base acumulado as técnicas que actúan sobre a consulta antes da recuperación.
6. **Experimento 5 (Reranking):** avalía a reordenación pos-recuperación dos candidatos sobre o mellor RAG base acumulado.
7. **Experimento SOTA (Validación):** compara a pipeline óptima obtida empiricamente cun *stack* que combina técnicas estado da arte da literatura, para situar o resultado nun marco de referencia externo.

As **variables independentes** son as configuracións específicas de cada experimento. As **variables dependentes** son as seis métricas definidas na Sección 3.5. As **variables controladas** son o modelo LLM xerador, o corpus documental e o golden dataset (construído na Sección 2.5), que permanecen constantes en todos os experimentos. O número de documentos recuperados fíxase en $\text{top}_k = 5$ salvo nas configuracións dos Experimentos 3 e 5, nos que se estuda explicitamente o efecto de k .

3.3. Arquitectura do sistema experimental

Toda a experimentación execútase sobre unha infraestrutura modular implementada con Kedro (Sección 1.6), organizada en cinco pipelines independentes que se encadean nun fluxo de datos único (Figura 3). Os tres primeiros —*ingestion*, *corpus_selection* e *gold_dataset*— prepáranse unha soa vez e permanecen constantes en todos os experimentos, garantindo que o corpus de traballo e o golden dataset son idénticos en todas as comparacións. Os pipelines *rag_base* (segmentación e indexación) e *evaluation* (execución da recuperación e a xeración e cálculo das seis métricas RAGAS) conforman o ciclo RAG, cuxa configuración se determina integramente por un ficheiro de parámetros (*parameters_exp*.yaml*), un por cada experimento

da ablación. Esta separación entre a lóxica e a configuración é a que permite que cada fase da ablación se reduza a cambiar parámetros, sen modificar código, e a que sustenta a reproducibilidade e a transferibilidade do marco a outros dominios.

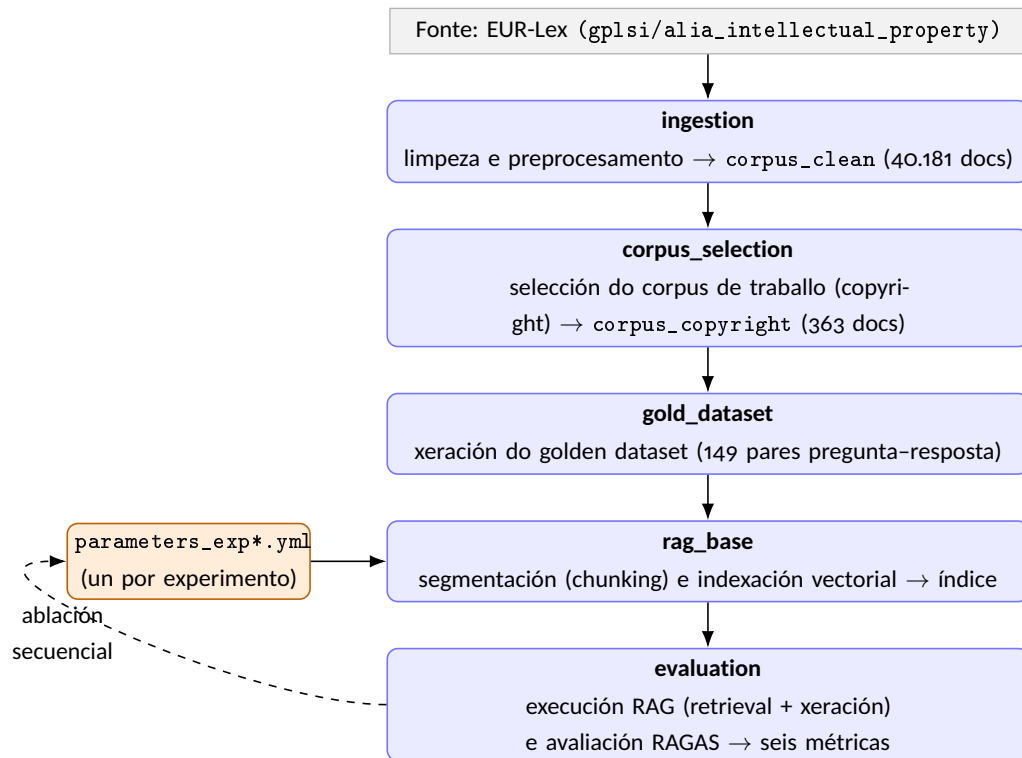


Figura 3: Arquitectura do sistema experimental: pipelines Kedro e fluxo de datos

Fonte: elaboración propia

A Táboa 5 resume a responsabilidade de cada pipeline e os artefactos que consome e produce, o que evidencia a modularidade do marco: cada etapa ten unha entrada e unha saída ben definidas e pode reexecutarse de forma illada.

Táboa 5: Pipelines Kedro: responsabilidade e artefactos de entrada e saída

Pipeline	Responsabilidade	Entrada → Saída
ingestion	Limpeza e preprocesamento do texto en bruto	raw_corpus → corpus_clean (40.181)
corpus_selection	Selección do corpus de traballo (dominio de copyright)	corpus_clean → corpus_copyright (363)
gold_dataset	Mostraxe, grafo de coñecemento e xeración de pares pregunta-resposta	corpus_copyright → gold_dataset (149)
rag_base	Segmentación (chunking) e indexación vectorial	corpus_copyright + parámetros → índice
evaluation	Execución RAG (recuperación + xeración) e avaliación RAGAS	índice + gold_dataset + parámetros → métricas

Fonte: elaboración propia

3.4. Técnicas avaliadas

3.4.1. Experimento 0: Liña base (Naive RAG)

O obxectivo do Experimento 0 é establecer un punto de referencia honesto: un sistema RAG mínimo cuxos scores documentan o punto de partida contra o que se miden todas as melloras posteriores. A configuración é:

- Corpus sen procesamento adicional máis alá da limpeza básica descrita na Sección 2.3.
- Chunking de tamaño fixo: 256 tokens (segundo o tokenizador do propio modelo de embedding), solapamento de 25 tokens (arredor dun 10 %).
- Modelo de embedding: BAAI/bge-base-en-v1.5 (Xiao et al., 2023) (86 M de parámetros, 768 dimensións, xanela de contexto de 512 tokens).
- Retriever: busca por similitude coseno, $\text{top}_k = 5$.
- Mesmo modelo LLM xerador para todos os experimentos.

Os scores obtidos gárdanse como `baseline_v0` e constitúen a referencia absoluta de comparación contra a que se mide cada mellora posterior.

3.4.2. Experimento 1: Comparativa de modelos de embedding

Obxectivo: identificar o modelo de embedding que mellor representa o corpus EUR-Lex e recupera os chunks máis relevantes para as consultas do golden dataset.

Variables fixas: corpus, chunking (tamaño fixo, 256 tokens, solapamento de 25 tokens), retriever cosine, $\text{top}_k = 5$, LLM, golden dataset.

Variable independente: modelo de embedding. Tómase como referencia o modelo do Experimento 0 (BAAI/bge-base-en-v1.5, Xiao et al. 2023) e compárase fronte a tres alternativas:

- BAAI/bge-m3 (Chen et al., 2024): modelo multilingüe da mesma familia BGE que xera simultaneamente representacións densas, esparsas e multi-vector, cunha xanela de contexto de até 8.192 tokens.
- Qwen/Qwen3-Embedding-8B (Zhang et al., 2025): modelo de embedding de 8.000 M de parámetros baseado na familia de LLM Qwen3, accedido mediante unha API de inferencia externa (Scaleway).
- Snowflake/snowflake-arctic-embed-l-v2.0 (Yu et al., 2024): modelo multilingüe optimizado para tarefas de recuperación, baseado en XLM-RoBERTa-large (568 M de parámetros).

Métricas de decisión: Context Recall e Context Precision, xa que o embedding afecta principalmente ao retriever. As demais métricas anótanse como observación para detectar patróns inesperados (por exemplo, un cambio de embedding que mellore Context Recall pero empeore Faithfulness indica un problema a investigar).

3.4.3. Experimento 2: Comparativa de estratexias de chunking

Obxectivo: identificar a estratexia de segmentación que mellor conserva o contexto relevante e encaixa coa estrutura formal dos documentos EUR-Lex.

Variables fixas: mellor embedding do Experimento 1, retriever cosine, $\text{top}_k = 5$, LLM, golden dataset. O tamaño de referencia pásase a 512 tokens, medidos co tokenizador do embedding gañador, con solapamento de 50 tokens onde a estratexia o permite.

Variable independente: estratexia de chunking. Avalíanse cinco estratexias:

- **Tamaño fixo** (`fixed_size`). `RecursiveCharacterTextSplitter` sobre o tokenizador do embedding gañador, con chunks de 512 tokens e solapamento de 50 tokens.
- **Por oracións** (`sentence`). Segmenta o texto en oracións e acumúlaas secuencialmente até acadar os 512 tokens, sen solapamento entre chunks consecutivos.
- **Estrutural** (`structural`). Segmenta seguindo a estrutura formal do documento: por artigos (identificados polo marcador “Article” seguido do número) nos actos lexislativos (regulamentos, directivas, decisións, etc.) e por parágrafos numerados nas resolucións xudiciais (sentenzas, conclusións, autos); as unidades resultantes que superan os 512 tokens subdivídense de forma recursiva, e os documentos sen marcadores estruturais recoñecibles segméntanse por parágrafos en branco.
- **Estrutural pai-fillo** (`structural_pc`). Igual á estratexia anterior, pero cada chunk (*fillo*, usado para indexar e recuperar) almacena tamén un *contexto pai* —a unión das unidades estruturais adxacentes até 1.024 tokens— que se entrega ao LLM xerador en lugar do chunk orixinal (Sección 1.4.5).
- **Semántico** (`semantic`). `SemanticChunker` (Kamradt, 2023): un modelo de embedding lixeiro local (`all-MiniLM-L6-v2`, Reimers e Gurevych 2019) detecta puntos de ruptura semántica por percentil de disimilitude entre oracións consecutivas; os segmentos resultantes que superan os 512 tokens subdivídense de forma recursiva.

Métricas de decisión: Context Recall (¿o chunk recuperado contén a resposta?) e Context Precision (¿os chunks recuperados son limpos e relevantes?).

3.4.4. Experimento 3: Comparativa de técnicas de retrieval

Obxectivo: avaliar que técnica de recuperación encontra mellor os chunks relevantes, usando o embedding e o chunking xa fixados.

Variables fixas: mellor embedding do Experimento 1, mellor chunking do Experimento 2, LLM, golden dataset.

Variable independente: técnica de retrieval e valor de top_k . As técnicas avalíanse en dous niveis:

- **Nivel 1** (referencia):
 - Busca por similitude coseno con $k \in \{3, 5, 10\}$ para illar o efecto do número de documentos recuperados.

- **Nivel 2** (recuperación alternativa, con $k \in \{5, 10\}$):
 - MMR (diversidade nos chunks devoltos).
 - BM25 (Robertson e Zaragoza, 2009) (recuperación esparsa pura).
 - Busca híbrida BM25 + vectorial con fusión por rango recíproco (Reciprocal Rank Fusion (RRF), Cormack et al. 2009).

Métricas de decisión: Context Recall, Context Precision e Noise Sensitivity. Esta última introdúcese como criterio de decisión neste experimento por primeira vez, xa que cuantifica directamente o impacto dos chunks irrelevantes sobre a xeración.

3.4.5. Experimento 4: Expansión e transformación de consultas

Obxectivo: avaliar se as técnicas que transforman ou expanden a consulta antes da recuperación producen melloras relevantes sobre o mellor RAG base acumulado.

Variables fixas: mellor configuración dos Experimentos 1–3, LLM, golden dataset. Cada técnica avalíase de forma independente.

- **HyDE:** o LLM xera un documento hipotético e este úsase como vector de busca (L. Gao et al., 2023).
- **Multi-Query:** xéranse $n = 3$ variantes da consulta e fúndense os resultados (Sección 1.4.3).
- **RAG-Fusion:** variantes da consulta combinadas mediante fusión por rango recíproco (RRF).
- **Reescrita de consultas** (*query rewriting*, Sección 1.4.3).
- **Step-Back:** o LLM xera unha pregunta máis xenérica para recuperar contexto de fondo.
- **Descomposición:** a consulta divídese en sub-preguntas que se recuperan de forma independente.
- **Self-Query:** extracción de filtros de metadatos a partir da consulta en linguaxe natural (Sección 1.4.4).

Métricas de decisión: Context Recall, Context Precision e Noise Sensitivity. As melloras de xeración (Faithfulness, Answer Relevancy) anótanse como evidencia secundaria.

3.4.6. Experimento 5: Reranking pos-recuperación

Obxectivo: avaliar se a reordenación dos candidatos recuperados mediante un *reranker* mellora a calidade do contexto e da resposta sobre o mellor RAG base acumulado.

Variables fixas: mellor configuración dos Experimentos 1–3, LLM, golden dataset.

Variable independente: tipo de reranker e valor de top_k . Aplícase unha cascada de dúas etapas: o retriever base recupera un *pool* de candidatos e un reranker reordénaos para reter os $\text{top}_k \in \{5, 10\}$ finais. Avalíanse dous tipos de reranker:

- **Cross-encoder:** BAAI/bge-reranker-v2-m3 (Chen et al., 2024), da mesma familia que o modelo de embedding bge-m3.

- **LLM listwise:** o propio LLM xerador recibe os candidatos nunha soa chamada e devolve a orde de relevancia.

Métricas de decisión: Context Recall, Context Precision, Noise Sensitivity e Faithfulness, dado que o reranking afecta tanto á calidade do contexto como á da xeración.

3.4.7. Experimento SOTA: validación contra un *stack* estado da arte

Obxectivo: situar a pipeline óptima obtida pola ablación secuencial nun marco de referencia externo, comparándoa cun *stack* maximalista que combina a técnica máis avanzada *sobre o papel* de cada dimensión —embedding Qwen3-Embedding-8B (líder en MTEB), chunking semántico, recuperación híbrida BM25 + vectorial con RRF, expansión de consultas RAG-Fusion e reranking listwise por LLM—, en lugar de seleccionar cada compoñente empiricamente fase a fase.

Variables fixas: LLM e golden dataset. **Comparación:** tres pipelines —o baseline (Naive RAG, Experimento 0), o *stack* SOTA da literatura e a pipeline empírica óptima (resultado da ablación)— avalíanse coas seis métricas RAGAS.

3.5. Métricas de avaliación

Empréganse seis métricas do framework RAGAS (definidas formalmente na Sección 1.5.1). As métricas clasifícanse como **primarias** (criterio de decisión no experimento correspondente) ou **secundarias** (rexistradas como observación en todos os experimentos para detectar efectos colaterais).

Context Recall. Métrica de recuperación. Mide en que medida o contexto recuperado contén toda a información necesaria para responder á pregunta. **Primaria** nos Experimentos 1–5.

Context Precision. Métrica de recuperación. Mide se os chunks relevantes están posicionados nos rangos superiores do contexto recuperado. **Primaria** nos Experimentos 1–5.

Context Entity Recall. Métrica de recuperación. Mide a proporción de entidades da resposta de referencia presentes no contexto recuperado. **Secundaria** en todos os experimentos.

Noise Sensitivity. Métrica de recuperación. Mide a proporción de afirmacións incorrectas na resposta atribuíbles a chunks irrelevantes no contexto. **Primaria** nos Experimentos 3, 4 e 5.

Faithfulness. Métrica de xeración. Mide en que medida as afirmacións da resposta están apoiadas polo contexto recuperado (Ecuación 2). **Primaria** no Experimento 5.

Response Relevancy. Métrica de xeración. Mide se a resposta xerada aborda a pregunta do usuario (Ecuación 3). **Secundaria** en todos os experimentos.

Todas as métricas RAGAS están normalizadas no intervalo $[0, 1]$. Como criterio de decisión agregado para as métricas de recuperación emprégase a suma $P+R = \text{Context Precision} + \text{Context Recall}$, que resume nun único valor a calidade global da recuperación de cada configuración.

A Táboa 6 resume o papel de cada métrica en cada experimento.

Táboa 6: Métricas de decisión (P) e de observación (S) por experimento

Métrica	Exp. 0	Exp. 1	Exp. 2	Exp. 3	Exp. 4	Exp. 5
Context Recall	S	P	P	P	P	P
Context Precision	S	P	P	P	P	P
Context Entity Recall	S	S	S	S	S	S
Noise Sensitivity	S	S	S	P	P	P
Faithfulness	S	S	S	S	S	P
Response Relevancy	S	S	S	S	S	S

P = métrica de decisión (criterio de elección da mellor configuración); S = métrica secundaria (rexistrada para observación).

Fonte: elaboración propia

4. Resultados e discusión

Nesta sección preséntanse os resultados da ablación secuencial descrita na Sección 3, avaliada sobre o golden dataset de 149 pares pregunta-resposta (75 *single-hop* e 74 *multi-hop*). Cada configuración aválase coas seis métricas RAGAS; como criterio agregado de decisión para a recuperación emprégase $P+R = \text{Context Precision} + \text{Context Recall}$. En todas as táboas, as columnas abreviadas son CR (Context Recall), CP (Context Precision), CER (Context Entity Recall), F (Faithfulness), RR (Response Relevancy) e NS (Noise Sensitivity); a fila gañadora de cada fase destácase en **negra**.

4.1. Experimento 1: modelos de embedding

A Táboa 7 recolle a comparativa dos catro modelos de embedding sobre a configuración base. O modelo `bge-m3` domina con claridade: eleva o $P+R$ desde 0,756 (baseline `bge-base-en-v1.5`) até 0,999, unha mellora absoluta de **+0,243**, a maior de toda a ablación. A mellora é transversal —Context Recall pasa de 0,453 a 0,583 e Response Relevancy de 0,423 a 0,561— o que confirma que a representación multilingüe e densa-esparsa-multivector de `bge-m3` captura mellor a semántica do corpus xurídico que os modelos máis pequenos. O modelo `qwen3-embedding-8b`, a pesar do seu tamaño (8.000 M de parámetros), queda por detrás ($P+R$ 0,889), o que indica que o tamaño do modelo non garante por si só unha mellor recuperación neste dominio.

Táboa 7: Resultados do Experimento 1 (modelos de embedding)

Configuración	CR	CP	CER	F	RR	NS	P+R
<code>bge-base-en-v1.5</code> (baseline)	0,453	0,304	0,324	0,833	0,423	0,390	0,756
<code>snowflake-arctic-embed-l</code>	0,463	0,312	0,325	0,843	0,456	0,359	0,775
<code>qwen3-embedding-8b</code>	0,502	0,387	0,308	0,866	0,456	0,353	0,889
<code>bge-m3</code> (gañador)	0,583	0,416	0,384	0,881	0,561	0,466	0,999

Fonte: elaboración propia

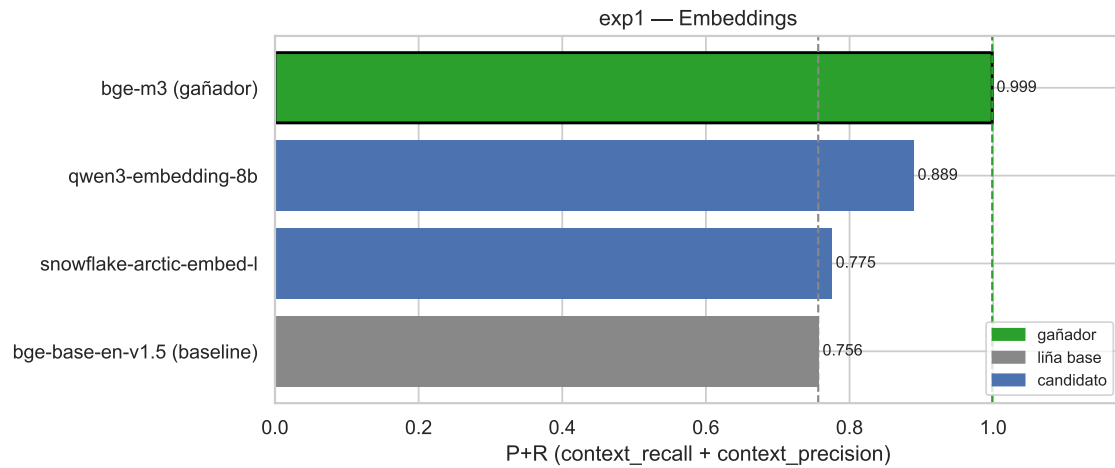


Figura 4: Comparativa de modelos de embedding (Experimento 1)

Fonte: elaboración propia

4.2. Experimento 2: estratexias de chunking

Fixado `bge-m3`, avaliáronse cinco estratexias de segmentación fronte ao chunking de tamaño fixo de 256 tokens da liña base (Táboa 8). O resultado é contraintuitivo: **ningunha estratexia supera o chunking fixo de 256 tokens** (P+R 0,999). A segmentación por oracións queda moi preto (0,993) e mellora o Context Recall (0,615), pero a un custo en Context Precision. A segmentación estrutural —que a priori encaixaría coa organización por artigos dos documentos EUR-Lex— resulta a peor opción (P+R 0,729), probablemente porque xera unidades de tamaño moi heteroxéneo que fragmentan o contexto relevante. En consecuencia, mantense o chunking fixo de 256 tokens.

Táboa 8: Resultados do Experimento 2 (estratexias de chunking)

Configuración	CR	CP	CER	F	RR	NS	P+R
fixed 256 (gañador = baseline)	0,583	0,416	0,384	0,881	0,561	0,466	0,999
fixed 512	0,580	0,333	0,356	0,805	0,479	0,425	0,913
sentence	0,615	0,378	0,362	0,870	0,460	0,405	0,993
structural	0,447	0,281	0,324	0,891	0,460	0,384	0,729
structural + parent	0,590	0,368	0,348	0,849	0,438	0,420	0,958
semantic	0,594	0,297	0,346	0,866	0,462	0,471	0,891

Fonte: elaboración propia

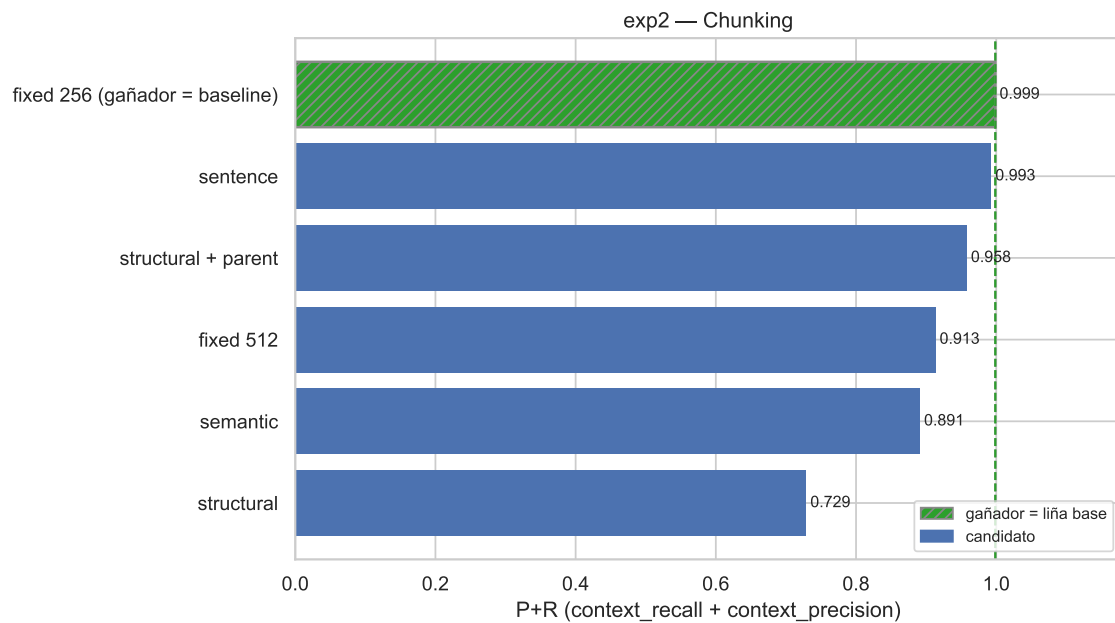


Figura 5: Comparativa de estratexias de chunking (Experimento 2)

Fonte: elaboración propia

4.3. Experimento 3: técnicas de retrieval

A Táboa 9 compara as técnicas de recuperación. Dúas conclusións son claras. Primeira, aumentar k de 5 a 10 mellora a recuperación densa (de 0,999 a 1,050 en P+R). Segunda, a diversificación MMR con $k = 10$ é a mellor opción (**P+R 1,094**, Context Recall 0,711), ao recuperar contextos máis variados que reducen a redundancia. As alternativas esparsa (BM25) e híbrida non superan a densa neste dominio: aínda que BM25 acada un Context Recall alto (0,656), a súa Context Precision cae drasticamente (0,258), o que penaliza o agregado. Adóptase, polo tanto, `mmr k10`.

Táboa 9: Resultados do Experimento 3 (técnicas de retrieval)

Configuración	CR	CP	CER	F	RR	NS	P+R
cosine k5 (baseline)	0,583	0,416	0,384	0,881	0,561	0,466	0,999
cosine k10	0,644	0,406	0,386	0,905	0,575	0,392	1,050
bm25 k10	0,656	0,258	0,384	0,826	0,403	0,409	0,914
hybrid k10	0,614	0,272	0,369	0,845	0,566	0,378	0,886
mmr k10 (gañador)	0,711	0,383	0,377	0,899	0,590	0,429	1,094

Fonte: elaboración propia

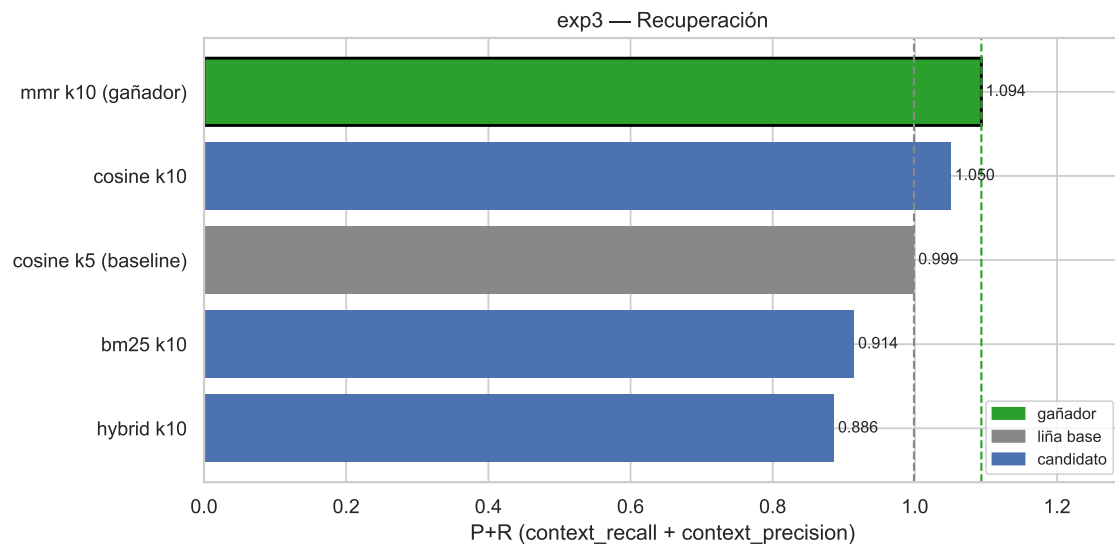


Figura 6: Comparativa de técnicas de retrieval (Experimento 3)

Fonte: elaboración propia

4.4. Experimento 4: expansión e transformación de consultas

Sobre a base `mmr k10` (P+R 1,094) avaliáronse sete técnicas de transformación da consulta (Táboa 10). O resultado é negativo: **ningunha técnica supera a base sen expansión**. A mellor, Multi-Query, queda lixeiramente por debaixo (1,047), e técnicas como HyDE (0,846) ou Self-Query (0,823) deterioran notablemente a recuperación. A interpretación é que, nun corpus xurídico onde as consultas xa son específicas e están ben formuladas, a expansión introduce ruído que desvía a recuperación máis do que a axuda. En consecuencia, a pipeline non incorpora ningunha técnica de expansión de consultas.

Táboa 10: Resultados do Experimento 4 (expansión de consultas)

Configuración	CR	CP	CER	F	RR	NS	P+R
mmr k10 (sen expansión)	0,711	0,383	0,377	0,899	0,590	0,429	1,094
Multi-Query	0,665	0,382	0,389	0,878	0,582	0,422	1,047
RAG-Fusion	0,650	0,365	0,359	0,918	0,515	0,387	1,015
Step-Back	0,654	0,349	0,341	0,895	0,544	0,339	1,003
Rewrite	0,633	0,354	0,354	0,938	0,568	0,408	0,987
Decomposition	0,660	0,283	0,383	0,919	0,593	0,420	0,942
HyDE	0,598	0,247	0,336	0,877	0,559	0,397	0,846
Self-Query	0,485	0,338	0,236	0,810	0,438	0,354	0,823

Fonte: elaboración propia

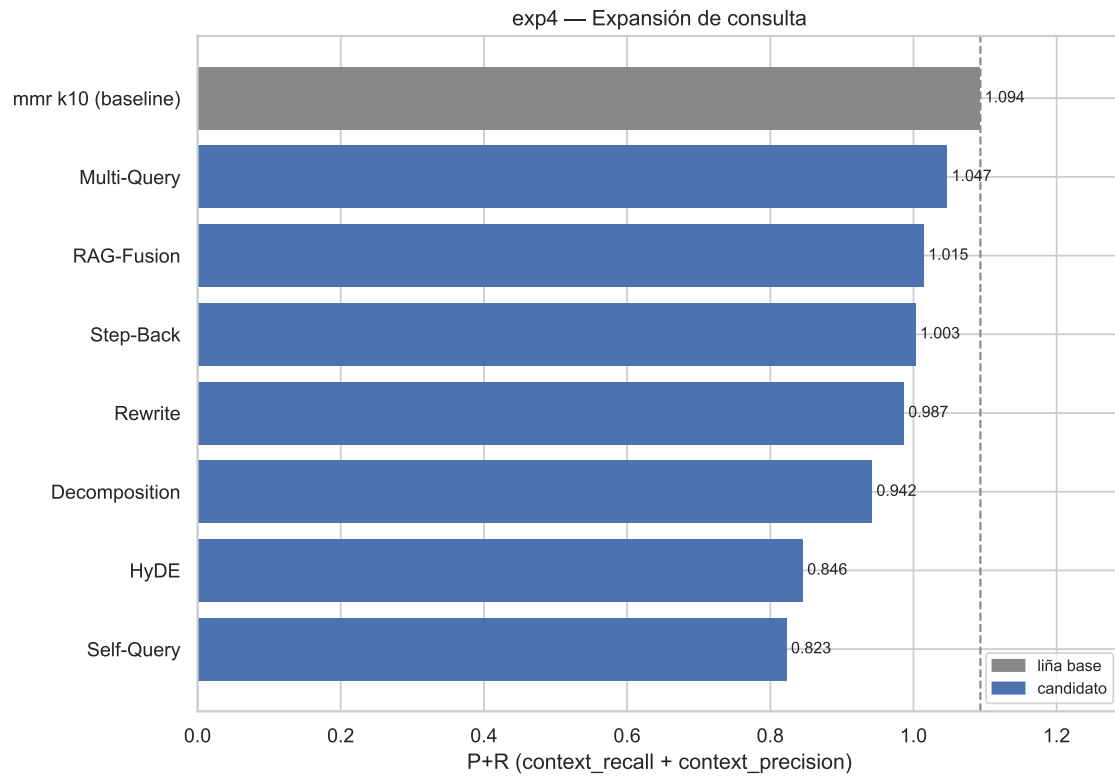


Figura 7: Comparativa de técnicas de expansión de consultas (Experimento 4)

Fonte: elaboración propia

4.5. Experimento 5: reranking pos-recuperación

A reordenación pos-recuperación é a segunda mellora máis importante da ablación (Táboa 11). O cross-encoder `bge-reranker-v2-m3` con $k = 10$ acadou o mellor resultado global do estudo: **P+R 1,203 (+0,109 sobre a base mmr k10)**, ademais do mellor Faithfulness (0,951) e a mellor Response Relevancy (0,650) de toda a experimentación. Os dous tipos de reranker melloran a base, pero o cross-encoder supera ao reranking listwise por LLM, cun custo computacional moi inferior (unha pasada do modelo cross-encoder fronte a unha chamada ao LLM xerador). Adóptase `cross-encoder k10` como último compoñente da pipeline.

Táboa 11: Resultados do Experimento 5 (reranking)

Configuración	CR	CP	CER	F	RR	NS	P+R
mmr k10 (sen rerank)	0,711	0,383	0,377	0,899	0,590	0,429	1,094
cross-encoder k5	0,708	0,479	0,374	0,924	0,556	0,403	1,187
LLM listwise k5	0,666	0,488	0,381	0,901	0,573	0,381	1,154
LLM listwise k10	0,721	0,463	0,441	0,890	0,579	0,432	1,184
cross-encoder k10 (gañador)	0,756	0,447	0,389	0,951	0,650	0,408	1,203

Fonte: elaboración propia

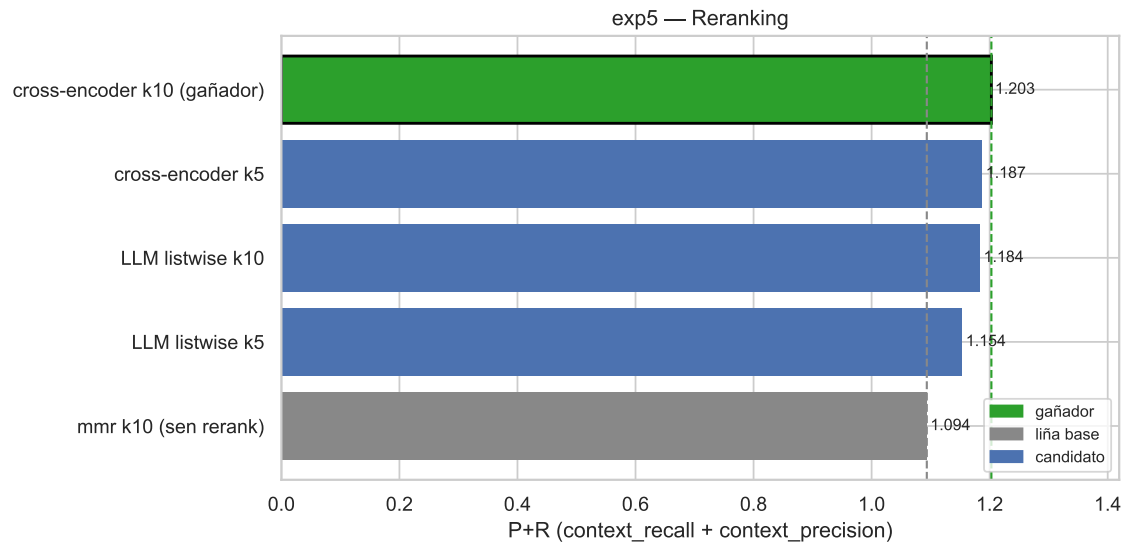


Figura 8: Comparativa de técnicas de reranking (Experimento 5)

Fonte: elaboración propia

4.6. Progresión global e pipeline final

A Táboa 12 e a Figura 9 resumen a progresión acumulada da ablación. A pipeline final óptima componse de: embedding bge-m3, chunking fixo de 256 tokens, recuperación mmr con $k = 10$, sen expansión de consultas, e reranking con cross-encoder bge-reranker-v2-m3 ($k = 10$). Esta configuración eleva o P+R desde 0,756 (baseline) até 1,203, unha mellora relativa do **59 %**. As tres decisións con impacto foron, por orde de magnitude, o embedding (+0,243), o reranking (+0,109) e o retrieval (+0,095); o chunking e a expansión de consultas non achegaron melloras.

Táboa 12: Progresión acumulada da ablación secuencial (métrica P+R)

Decisión acumulada	P+R	Δ
Baseline (Exp. o)	0,756	—
+ Embedding: bge-m3	0,999	+0,243
+ Chunking: fixed 256	0,999	+0,000
+ Retrieval: mmr k10	1,094	+0,095
+ Expansión de consultas: —	1,094	+0,000
+ Reranking: ce k10	1,203	+0,109

Fonte: elaboración propia

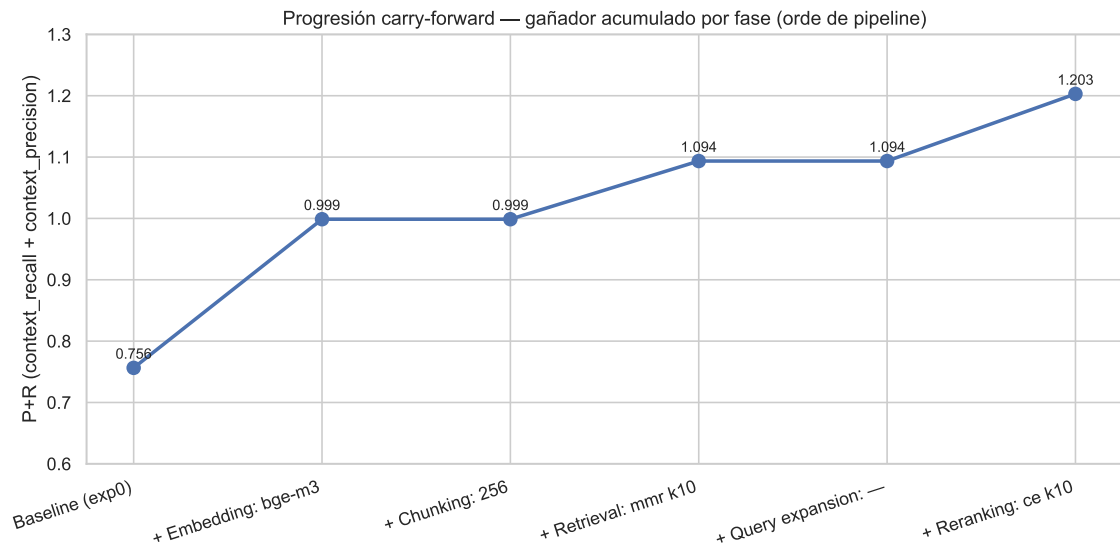


Figura 9: Progresión acumulada do P+R ao longo da ablación secuencial

Fonte: elaboración propia

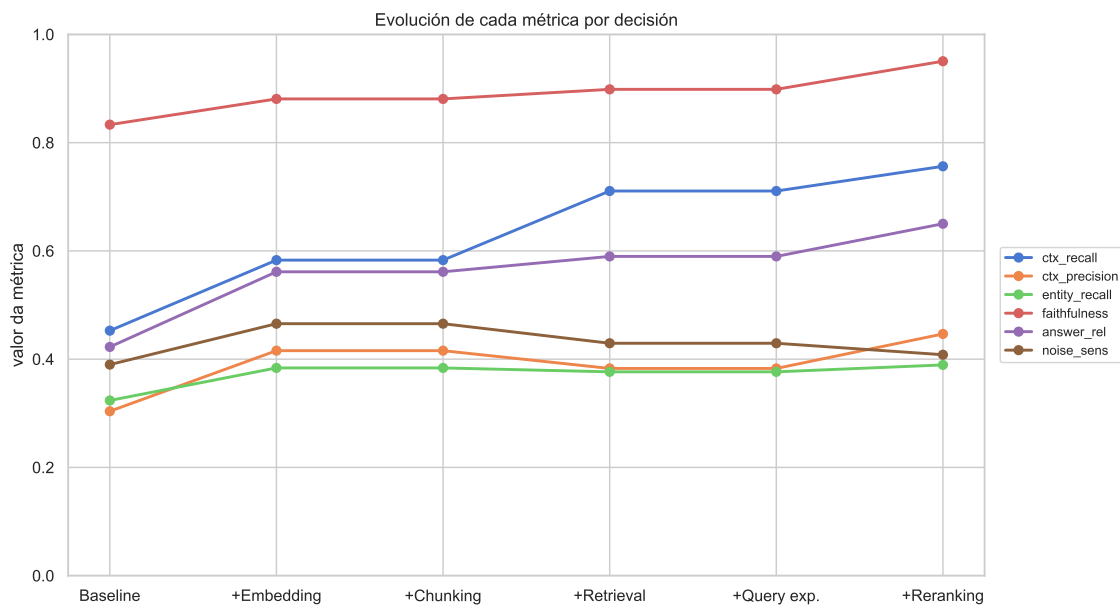


Figura 10: Evolución das seis métricas RAGAS ao longo da ablación

Fonte: elaboración propia

A comparación directa entre o baseline e a pipeline final (Táboa 13) mostra melloras en todas as métricas de decisión: Context Recall (+0,303), Context Precision (+0,143), Faithfulness (+0,118) e Response Relevancy (+0,227).

Táboa 13: Baseline fronte á pipeline final óptima

Configuración	CR	CP	CER	F	RR	NS	P+R
Baseline (Exp. o)	0,453	0,304	0,324	0,833	0,423	0,390	0,756
Pipeline final (ce k10)	0,756	0,447	0,389	0,951	0,650	0,408	1,203

Fonte: elaboración propia

4.7. Validación contra un *stack* estado da arte

Finalmente, a pipeline óptima compárase cun *stack* maximalista que combina a técnica máis avanzada sobre o papel de cada dimensión —*Qwen3-Embedding-8B*, chunking semántico, recuperación híbrida, RAG-Fusion e reranking listwise por LLM— (Táboa 14, Figura 11). O resultado valida o enfoque empírico: a pipeline obtida pola ablación secuencial (P+R 1,203) **supera** tanto o baseline (0,756) como o *stack* SOTA da literatura (1,095), cun mellor Context Precision, Faithfulness e Response Relevancy. O *stack* SOTA só aventaxa a pipeline empírica en Noise Sensitivity (0,486 fronte a 0,408). Isto confirma que a selección empírica de compoñentes adaptada ao dominio supera a aplicación directa de técnicas xenéricas consideradas estado da arte.

Táboa 14: Pipeline empírica fronte ao *stack* SOTA da literatura

Configuración	CR	CP	CER	F	RR	NS	P+R
Naive RAG (Exp. 0)	0,453	0,304	0,324	0,833	0,423	0,390	0,756
Stack SOTA (literatura)	0,708	0,387	0,354	0,869	0,566	0,486	1,095
Pipeline empírica	0,756	0,447	0,389	0,951	0,650	0,408	1,203

Fonte: elaboración propia

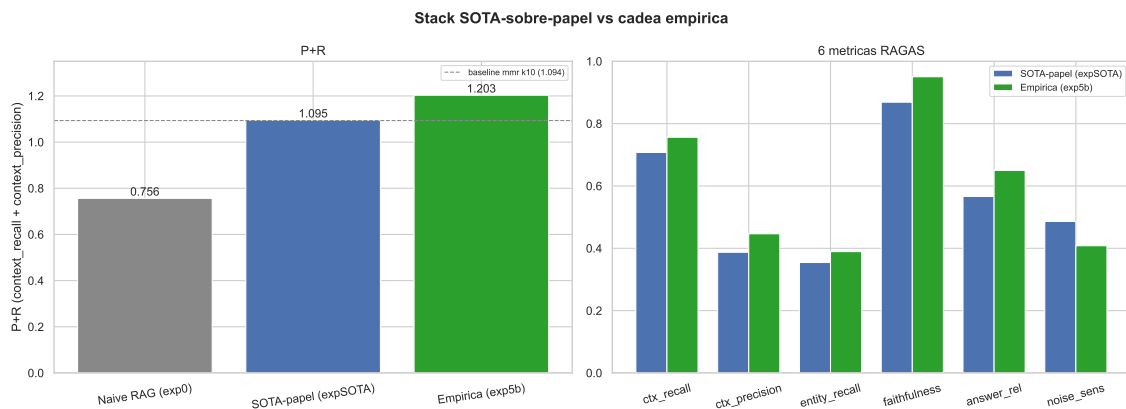


Figura 11: Comparación da pipeline empírica co *stack* SOTA e o baseline

Fonte: elaboración propia

Conclusións e ampliación

Síntese de resultados

Este traballo abordou a optimización experimental dunha pipeline RAG sobre o corpus xurídico EUR-Lex mediante unha ablación secuencial de cinco fases, avaliada con seis métricas RAGAS sobre un golden dataset de 149 pares pregunta-resposta. A configuración final óptima —embedding `bge-m3`, chunking fixo de 256 tokens, recuperación `mmr` con $k = 10$, sen expansión de consultas e reranking con cross-encoder `bge-reranker-v2-m3` ($k = 10$)— elevou a métrica agregada de recuperación P+R desde 0,756 (baseline Naive RAG) até 1,203, unha mellora relativa do 59 %, e superou tamén a un *stack* que combina técnicas estado da arte da literatura (P+R 1,095).

As preguntas de investigación respóndense do seguinte xeito:

- RQ1:** O modelo `bge-m3` é o que mellor representa o corpus EUR-Lex, cunha mellora de +0,243 en P+R sobre o baseline —a maior decisión individual de toda a ablación—. O tamaño do modelo non é determinante: o moito maior `qwen3-embedding-8b` obtivo peores resultados.
- RQ2:** Ningunha estratexia de chunking avanzada superou o chunking fixo de 256 tokens. A segmentación estrutural, a priori axeitada para documentos xurídicos, resultou a peor opción, polo que se mantivo o chunking fixo.
- RQ3:** A diversificación MMR con $k = 10$ mellorou de forma consistente a recuperación densa (+0,095 en P+R). A recuperación esparsa (BM25) e a híbrida non superaron a densa neste dominio.
- RQ4:** Ningunha das sete técnicas de expansión de consultas mellorou a base; en cambio, o reranking con cross-encoder ($k = 10$) foi a segunda mellora máis importante (+0,109 en P+R), superando ao reranking listwise por LLM cun custo computacional moi inferior.
- RQ5:** A pipeline construída empiricamente mediante a ablación secuencial superou o *stack* de técnicas estado da arte da literatura, o que confirma que a selección de compoñentes adaptada ao dominio é máis eficaz que a aplicación directa de técnicas xenéricas.

Implicación metodolóxica

Máis alá das configuracións concretas gañadoras, o achado transversal deste traballo é de natureza metodolóxica: **non existe unha única receita correcta para construír unha pipeline RAG**. A configuración óptima depende do dominio, da natureza dos documentos e do tipo de consultas, e só pode determinarse de forma empírica. Esta afirmación non é unha intuición, senón que se sostén directamente sobre a evidencia recollida ao longo da ablación:

- O *stack* que combina as técnicas consideradas estado da arte na literatura —o que cabería esperar como “a forma correcta” de facelo— foi superado pola pipeline axustada empiricamente fase a fase (P+R 1,095 fronte a 1,203).

- Decisións aparentemente axeitadas para o dominio fallaron: a segmentación estrutural, deseñada para aproveitar a organización por artigos dos textos xurídicos, resultou a peor estratexia de chunking.
- Técnicas amplamente recomendadas non aportaron ningunha mellora: ningunha das sete técnicas de expansión de consultas superou a base.
- O tamaño non garante o rendemento: o modelo de embedding máis grande (Qwen3-Embedding-8B, 8.000 M de parámetros) foi superado por un modelo moito máis pequeno (bge-m3).

En consecuencia, a principal recomendación práctica deste traballo é que a única forma correcta de optimizar un sistema RAG é probalo empiricamente sobre o dominio concreto de aplicación: a optimización debe abordarse sempre como un proceso experimental e medido, en lugar de asumir que as técnicas máis avanzadas ou de maior tamaño producirán por si soas os mellores resultados. Esta conclusión é coherente coa marcada variabilidade do rendemento dos sistemas RAG entre dominios sinalada na literatura (Y. Gao et al., 2023).

Contribución instrumental e aplicabilidade

A implicación anterior outorga un valor especial ao segundo resultado deste traballo, de natureza instrumental. Se a configuración óptima dun sistema RAG non é universal e debe determinarse empiricamente en cada dominio, entón o activo máis duradeiro non son os gañadores concretos —que envellecen a medida que aparecen novos modelos— senón o **marco experimental reproducible** construído para obtelos. Este traballo materializa ese marco como unha infraestrutura parametrizable (implementada con Kedro, Sección 1.6) na que cada experimento se define por configuración, é reproducible e auditable, e na que substituír o corpus, o modelo de embedding ou calquera outro compoñente non require reescribir código, senón cambiar parámetros.

Esta separación entre lóxica e configuración converte o marco nun entregable reutilizable con aplicabilidade directa en contornos profesionais: calquera organización que dispoña dun corpus documental propio pode empregalo para identificar, de forma sistemática e medida, a configuración RAG máis axeitada ao seu dominio antes de levala a produción, e re-executalo periodicamente a medida que evolucionan os modelos dispoñibles. Deste xeito, a contribución do traballo non se limita a un resultado empírico puntual sobre EUR-Lex, senón que ofrece un método e unha ferramenta transferibles para a toma de decisións tecnolóxicas no deseño de sistemas RAG.

Limitacións do estudo

O presente estudo presenta as seguintes limitacións que deben terse en conta ao interpretar os resultados:

- **Dominio único.** Os experimentos realízanse exclusivamente sobre documentos xurídicos europeos. A xeneralización dos resultados a outros dominios (médico, financeiro, científico) non está garantida.

- **Custo computacional de RAGAS.** A avaliación con RAGAS require múltiples chamadas ao LLM por exemplo, o que implica un custo económico e limita o número de configuracións avaliábeis.
- **Dependencia do modelo avaliador.** RAGAS usa un LLM como xuíz, polo que os resultados das métricas están parcialmente condicionados pola calidade e os sesgos do modelo empregado para a avaliación.
- **Golden dataset limitado.** O tamaño do conxunto de avaliación está limitado polo custo de construción manual. Conxuntos de maior tamaño poderían reducir a varianza nas estimacións das métricas.
- **Alcance das arquitecturas avaliadas.** O estudo céntrase na optimización dos compoñentes dunha pipeline RAG estándar (embedding, chunking, retrieval, expansión de consultas e reranking). As arquitecturas que modifican o fluxo completo do sistema (CRAG, Adaptive-RAG, RAG axéntico, GraphRAG) quedaron fóra do alcance experimental polo seu elevado custo de implementación e execución, e propóñense como traballo futuro.
- **Optimización greedy.** A ablación secuencial non garante un óptimo global: posibles interaccións entre compoñentes (por exemplo, entre embedding e chunking) poderían facer que outra combinación non explorada superase a obtida. Avaliar exhaustivamente o produto cartesiano de todas as opcións consideradas en cada fase —catro modelos de embedding, seis estratexias de chunking, oito configuracións de retrieval, oito de expansión de consultas e cinco de reranking— implicaría arredor de 7.680 combinacións ($4 \times 6 \times 8 \times 8 \times 5$), cada unha cun custo de avaliación RAGAS proporcional ao tamaño do golden dataset. Tal busca exhaustiva é computacionalmente inviable, polo que a ablación secuencial —que reduce o problema a unhas poucas decenas de configuracións (arredor de 25)— constitúe un compromiso necesario entre exhaustividade e custo.

Traballo futuro

Este traballo abre varias liñas de investigación futura:

- Extensión do benchmark a outros dominios especializados (medicina, finanzas) para contrastar a xeneralizabilidade dos resultados obtidos en EUR-Lex.
- Estudo do impacto de modelos de embedding específicos do dominio xurídico (*fine-tuned* sobre corpus legal) sobre as métricas de recuperación, máis alá dos modelos de propósito xeral avaliados neste traballo.
- Avaliación de arquitecturas RAG que modifican o fluxo completo do sistema —Self-RAG, RAG correctivo (CRAG), Adaptive-RAG, RAG axéntico con LangGraph e GraphRAG— sobre a pipeline óptima obtida neste traballo, para comprobar se superan ao mellor RAG base.
- Avaliación de estratexias de recuperación iterativa activa como FLARE, que decide adaptativamente cando xerar e cando recuperar información adicional.

- Construción dun golden dataset de maior tamaño mediante anotación humana sistemática, co obxectivo de reducir a varianza nas estimacións das métricas e permitir análises estatísticas máis robustas.
- Análise do impacto do modelo LLM xerador sobre os resultados, comparando modelos de código aberto con modelos de propósito xeral para identificar opcións economicamente viables en produción.
- Evolución do marco experimental cara a unha ferramenta de produción completa: despregamento do sistema RAG óptimo como servizo (API e interface de usuario), medición de latencia e custo en operación real, e integración do proceso de optimización nun fluxo de *MLOps* que permita reavaliar a configuración de forma automática a medida que evolucionan os modelos e o corpus.

Bibliografía

- Asai, Akari, Zeqiu Wu, Yizhong Wang, Avirup Sil e Hannaneh Hajishirzi (2024). “Self-RAG: Learning to retrieve, generate, and critique through self-reflection”. En: *The Twelfth International Conference on Learning Representations*. URL: <https://arxiv.org/abs/2310.11511>.
- Brown, Tom B., Benjamin Mann, Nick Ryder et al. (2020). “Language models are few-shot learners”. En: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., pp. 1877–1901. URL: <https://arxiv.org/abs/2005.14165>.
- Chase, Harrison (2022). *LangChain: Building applications with large language models*. Software de código aberto. URL: <https://github.com/langchain-ai/langchain> (accedido en 01/12/2024).
- Chen, Jianlv, Shitao Xiao, Peitian Zhang, Kun Luo, Defu Lian e Zheng Liu (2024). *M3-Embedding: Multi-Linguality, Multi-Functionality, Multi-Granularity Text Embeddings Through Self-Knowledge Distillation*. arXiv preprint arXiv:2402.03216. URL: <https://arxiv.org/abs/2402.03216>.
- Cormack, Gordon V., Charles L. A. Clarke e Stefan Buettcher (2009). “Reciprocal rank fusion outperforms Condorcet and individual rank learning methods”. En: *Proceedings of the 32nd International ACM SIGIR Conference on Research and Development in Information Retrieval*. ACM, pp. 758–759. DOI: [10.1145/1571941.1572114](https://doi.org/10.1145/1571941.1572114).
- Devlin, Jacob, Ming-Wei Chang, Kenton Lee e Kristina Toutanova (2019). “BERT: Pre-training of deep bidirectional transformers for language understanding”. En: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies, Volume 1 (Long and Short Papers)*. Minneapolis, Minnesota: Association for Computational Linguistics, pp. 4171–4186. DOI: [10.18653/v1/N19-1423](https://doi.org/10.18653/v1/N19-1423). URL: <https://aclanthology.org/N19-1423>.
- Edge, Darren, Ha Trinh, Newman Cheng et al. (2024). *From Local to Global: A Graph RAG Approach to Query-Focused Summarization*. arXiv preprint arXiv:2404.16130. URL: <https://arxiv.org/abs/2404.16130>.
- Es, Shahul, Jithin James, Luis Espinosa-Anke e Steven Schockaert (2023). *RAGAS: Automated evaluation of retrieval augmented generation*. arXiv preprint arXiv:2309.15217. URL: <https://arxiv.org/abs/2309.15217>.
- Espinosa Zaragoza, Sergio, María Miró Maestre, Rafael Muñoz Guillena e Juan Pablo Consuegra-Ayala (2025). *ALIA_INTELLECTUAL_PROPERTY Dataset*. https://huggingface.co/datasets/gplsi/alia_intellectual_property. Financiado polo Ministerio para la Transformación Digital y de la Función Pública, cofinanciado pola UE – NextGenerationEU, proxecto ALIA. Licenza: CC BY 4.0.

- Gao, Luyu, Xueguang Ma, Jimmy Lin e Jamie Callan (2023). “Precise zero-shot dense retrieval without relevance labels”. En: *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*. Association for Computational Linguistics, pp. 1762–1777. DOI: [10.18653/v1/2023.acl-long.99](https://doi.org/10.18653/v1/2023.acl-long.99). URL: <https://aclanthology.org/2023.acl-long.99>.
- Gao, Yunfan, Yun Xiong, Xinyu Gao et al. (2023). *Retrieval-augmented generation for large language models: A survey*. arXiv preprint arXiv:2312.10997. URL: <https://arxiv.org/abs/2312.10997>.
- Jeong, Soyeong, Jinheon Baek, Sukmin Cho, Sung Ju Hwang e Jong C. Park (2024). *Adaptive-RAG: Learning to Adapt Retrieval-Augmented Large Language Models through Question Complexity*. arXiv preprint arXiv:2403.14403. URL: <https://arxiv.org/abs/2403.14403>.
- Ji, Ziwei, Nayeon Lee, Rita Frieske et al. (2023). “Survey of hallucination in natural language generation”. En: *ACM Computing Surveys* 55.12, pp. 1–38. DOI: [10.1145/3571730](https://doi.org/10.1145/3571730).
- Kamradt, Greg (2023). *Semantic chunking for RAG*. FullStackRetrieval-com/RetrievalTutorials. URL: <https://github.com/FullStackRetrieval-com/RetrievalTutorials> (accedido en 01/12/2024).
- Karpukhin, Vladimir, Barlas Oguz, Sewon Min et al. (2020). “Dense passage retrieval for open-domain question answering”. En: *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, pp. 6769–6781. DOI: [10.18653/v1/2020.emnlp-main.550](https://doi.org/10.18653/v1/2020.emnlp-main.550). URL: <https://aclanthology.org/2020.emnlp-main.550>.
- Lewis, Patrick, Ethan Perez, Aleksandra Piktus et al. (2020). “Retrieval-augmented generation for knowledge-intensive NLP tasks”. En: *Advances in Neural Information Processing Systems*. Vol. 33. Curran Associates, Inc., pp. 9459–9474. URL: <https://arxiv.org/abs/2005.11401>.
- Malkov, Yu A. e D. A. Yashunin (2020). “Efficient and Robust Approximate Nearest Neighbor Search Using Hierarchical Navigable Small World Graphs”. En: *IEEE Transactions on Pattern Analysis and Machine Intelligence* 42.4, pp. 824–836. DOI: [10.1109/TPAMI.2018.2889473](https://doi.org/10.1109/TPAMI.2018.2889473).
- Nogueira, Rodrigo e Kyunghyun Cho (2019). *Passage re-ranking with BERT*. arXiv preprint arXiv:1901.04085. URL: <https://arxiv.org/abs/1901.04085>.
- Publications Office of the European Union (2024). *EUR-Lex: Access to European Union law*. URL: <https://eur-lex.europa.eu> (accedido en 01/12/2024).
- Qdrant (2024). *Qdrant: Vector Database & Search Engine*. Software de código aberto. URL: <https://github.com/qdrant/qdrant> (accedido en 01/12/2024).
- QuantumBlack, AI by McKinsey (2023). *Kedro: A Python Framework for Reproducible, Maintainable and Modular Data Science Code*. Software de código aberto. URL: <https://kedro.org> (accedido en 27/05/2025).

- Reimers, Nils e Iryna Gurevych (2019). "Sentence-BERT: Sentence embeddings using siamese BERT-networks". En: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. Association for Computational Linguistics, pp. 3982–3992. DOI: [10.18653/v1/D19-1410](https://doi.org/10.18653/v1/D19-1410). URL: <https://aclanthology.org/D19-1410>.
- Robertson, Stephen e Hugo Zaragoza (2009). "The probabilistic relevance framework: BM25 and beyond". En: *Foundations and Trends® in Information Retrieval* 3.4, pp. 333–389. DOI: [10.1561/15000000019](https://doi.org/10.1561/15000000019).
- Shi, Jiale et al. (2024). *A systematic analysis of chunking strategies for reliable question answering*. arXiv preprint. VERIFICAR: identificador arXiv e lista completa de autores.
- Vaswani, Ashish, Noam Shazeer, Niki Parmar et al. (2017). "Attention is all you need". En: *Advances in Neural Information Processing Systems*. Vol. 30. Curran Associates, Inc. URL: <https://arxiv.org/abs/1706.03762>.
- Wang, Liang, Nan Yang, Xiaolong Huang et al. (2022). *Text Embeddings by Weakly-Supervised Contrastive Pre-Training*. arXiv preprint arXiv:2212.03533. URL: <https://arxiv.org/abs/2212.03533>.
- Xiao, Shitao, Zheng Liu, Peitian Zhang e Niklas Muennighoff (2023). *C-Pack: Packaged Resources To Advance General Chinese Embedding*. arXiv preprint arXiv:2309.07597. URL: <https://arxiv.org/abs/2309.07597>.
- Yan, Shi-Qi, Jia-Chen Gu, Yun Zhu e Zhen-Hua Ling (2024). *Corrective Retrieval Augmented Generation*. arXiv preprint arXiv:2401.15884. URL: <https://arxiv.org/abs/2401.15884>.
- Yu, Puxuan, Luke Merrick, Gaurav Nuti e Daniel Campos (2024). *Arctic-Embed 2.0: Multilingual Retrieval Without Compromise*. arXiv preprint arXiv:2412.04506. URL: <https://arxiv.org/abs/2412.04506>.
- Zhang, Yanzhao, Mingxin Li, Dingkun Long et al. (2025). *Qwen3 Embedding: Advancing Text Embedding and Reranking Through Foundation Models*. arXiv preprint arXiv:2506.05176. URL: <https://arxiv.org/abs/2506.05176>.